



Review Class

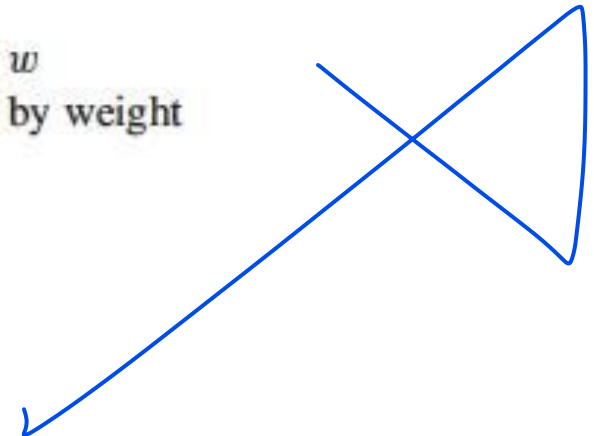
Summary of all the chapters

Consider the Following Sets

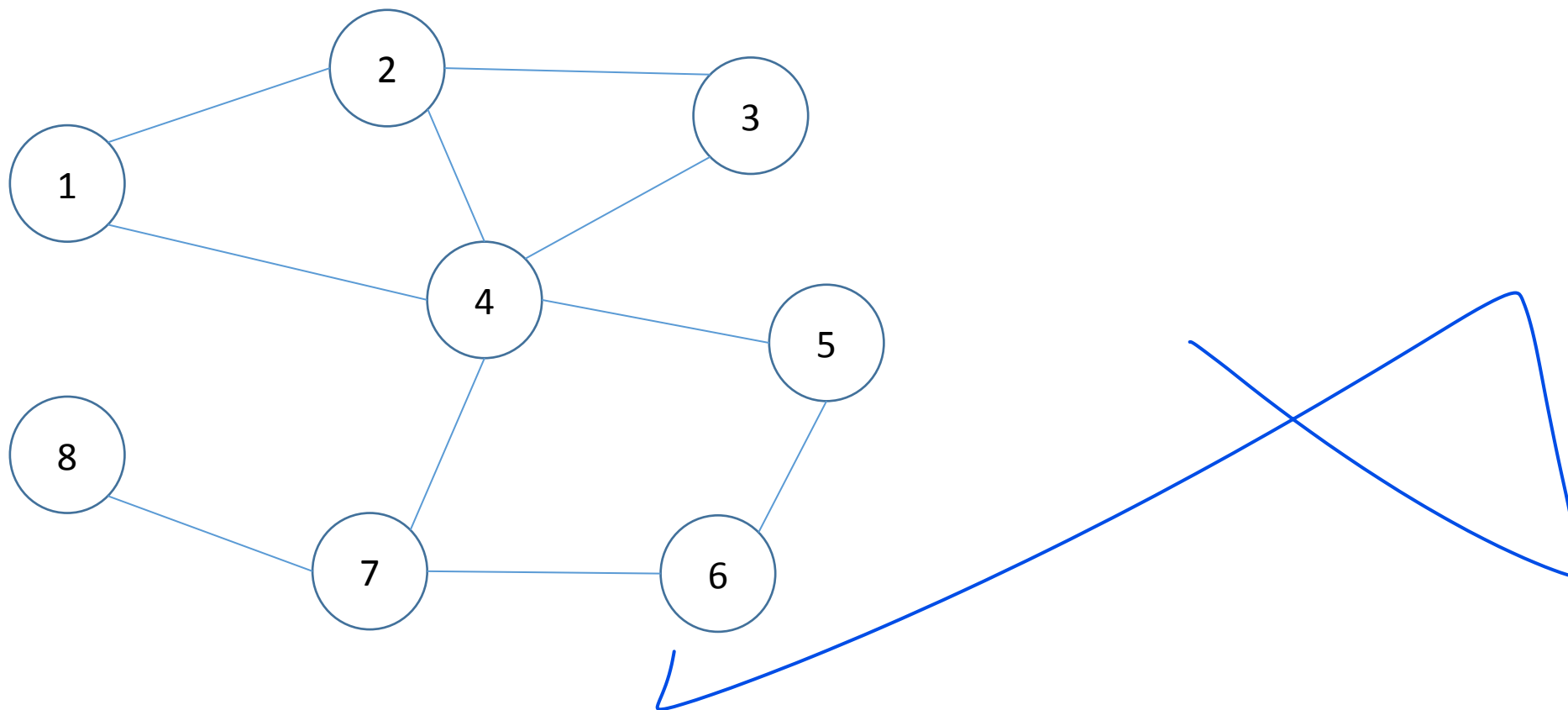
- Why is sets relevant here?
 - Set is required for different mathematical operations
 - Set is an essential data structure for **Kruskal's Algorithm**
 - Set can be used for graph in different ways

MST-KRUSKAL(G, w)

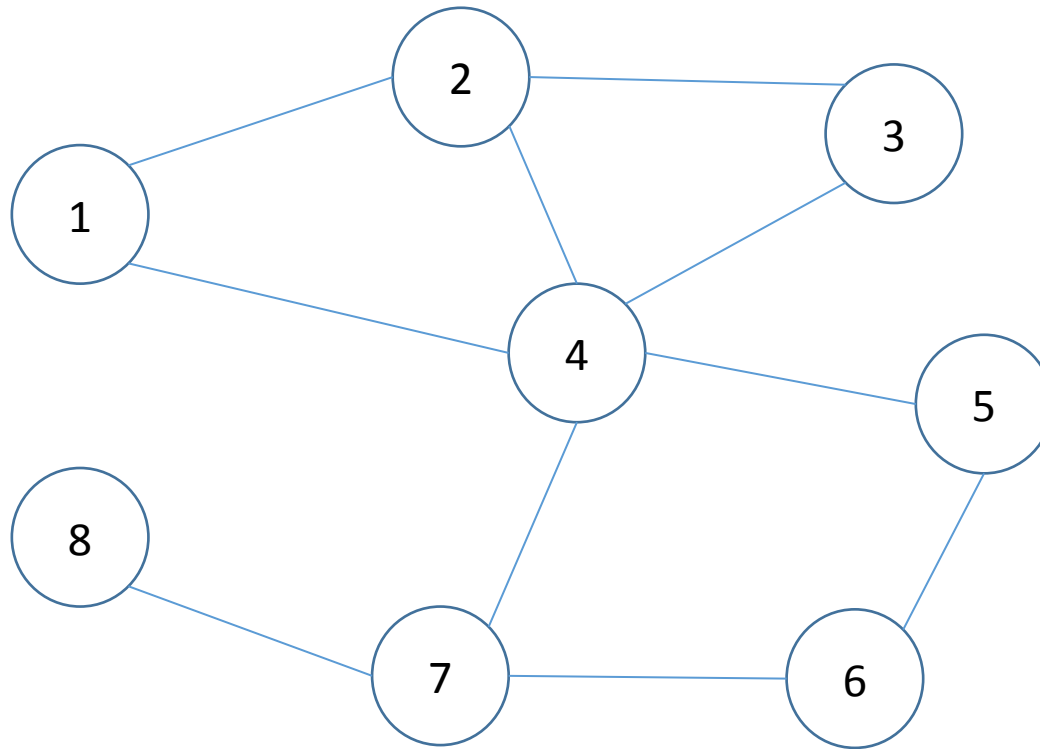
```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```



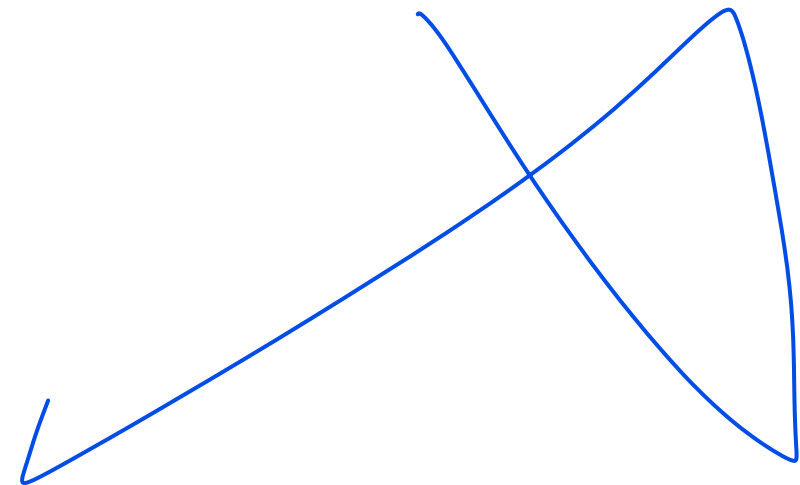
Consider the Following Graph



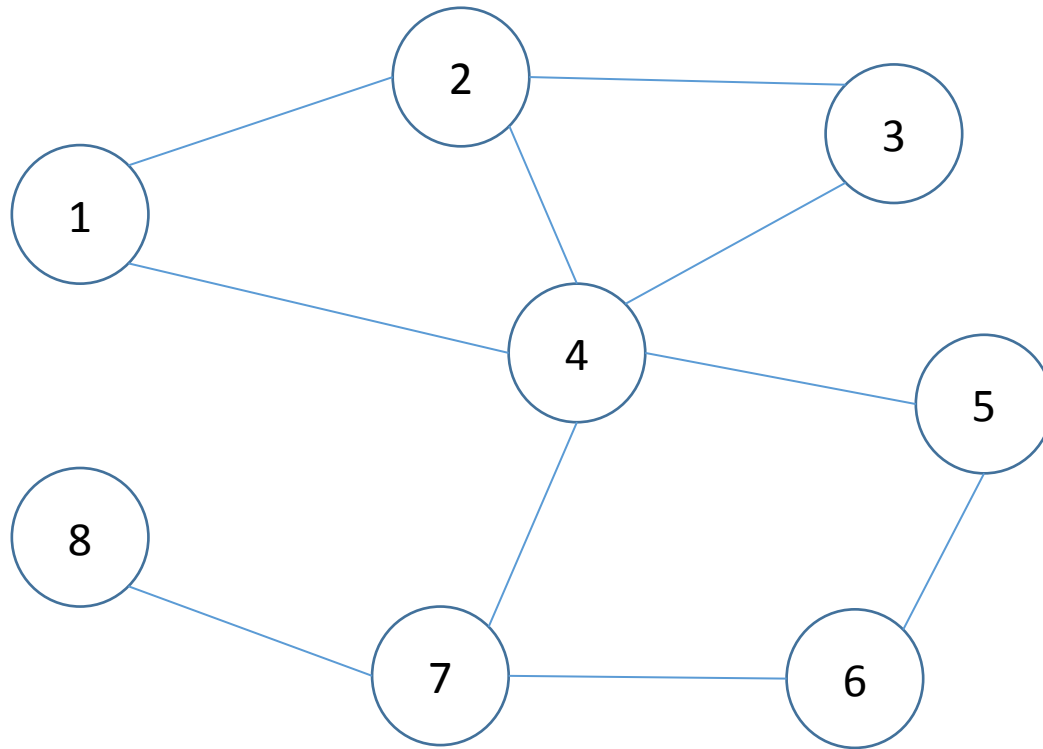
Consider the Following Graph



Let us the group the vertices
into two sets



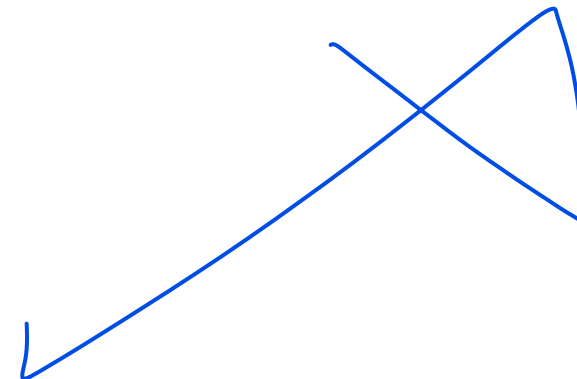
Consider the Following Graph



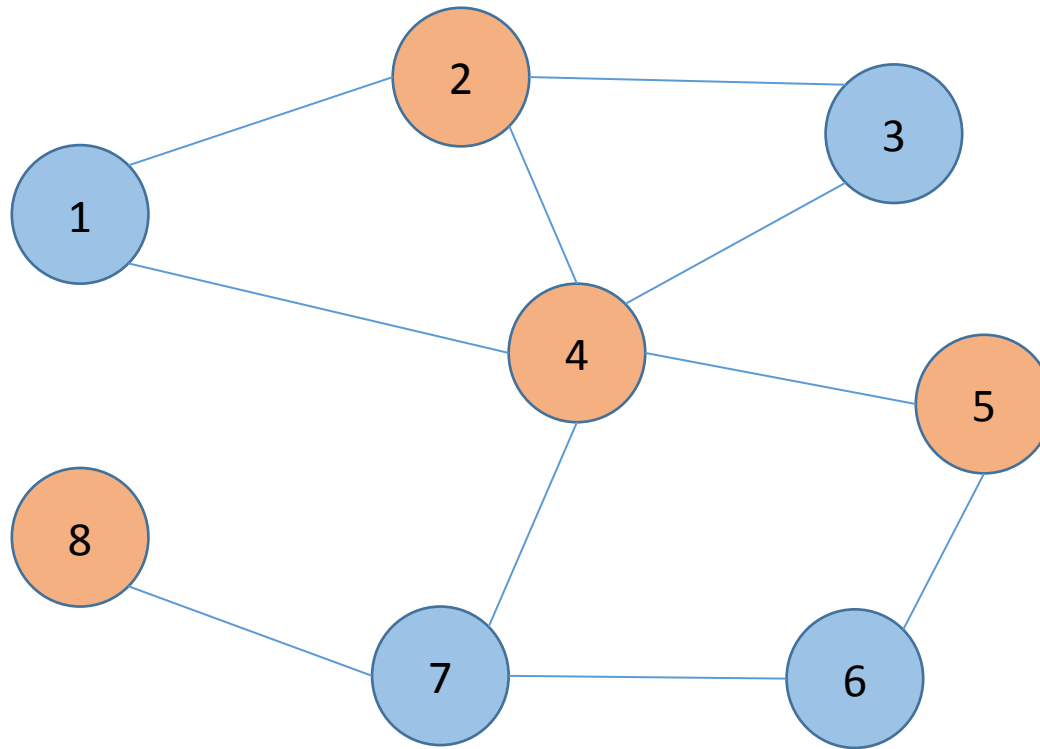
Let us group the vertices
into two sets

$$A = \{1, 3, 6, 7\}$$

$$B = \{2, 4, 5, 8\}$$



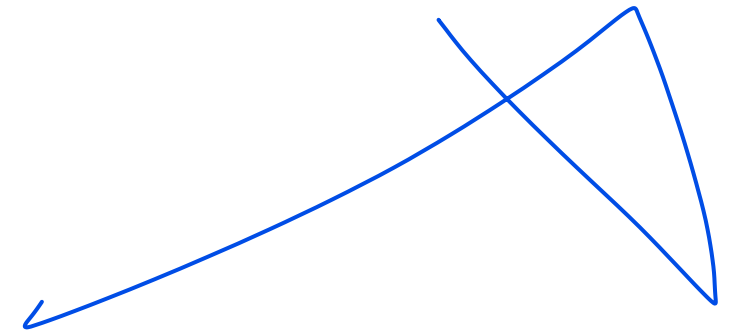
Consider the Following Graph



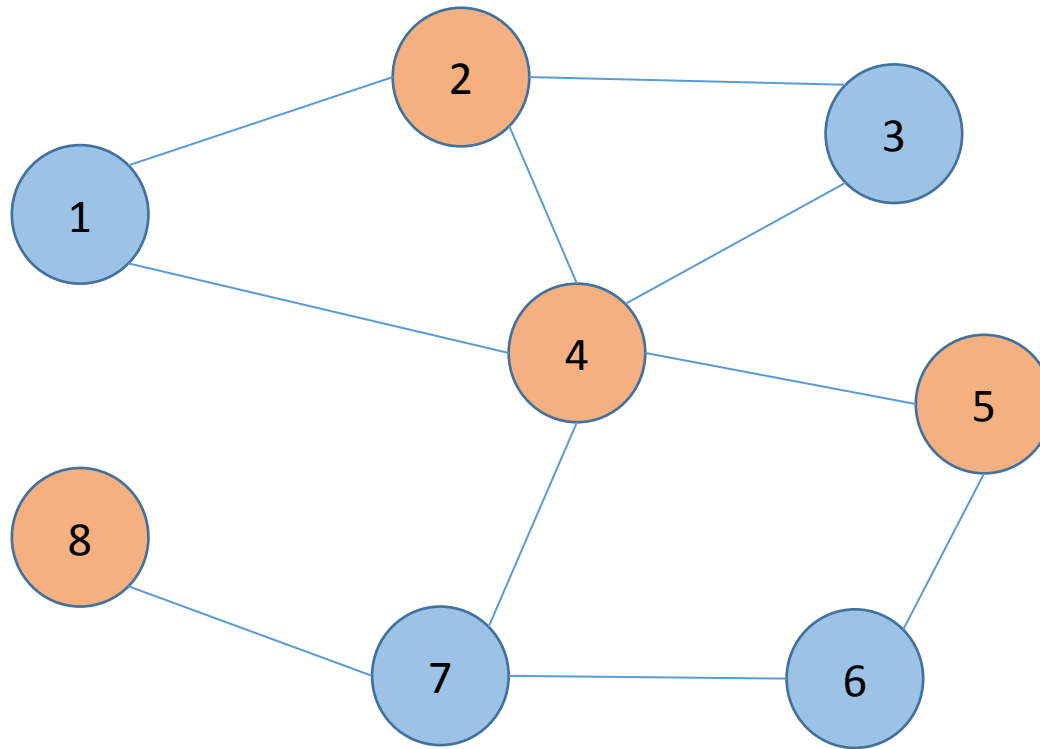
Let us group the vertices into two sets

$$A = \{1, 3, 6, 7\}$$

$$B = \{2, 4, 5, 8\}$$



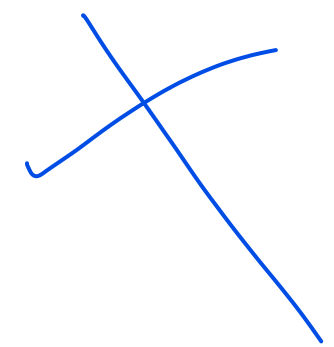
Consider the Following Graph



Let us the group the vertices
into two sets

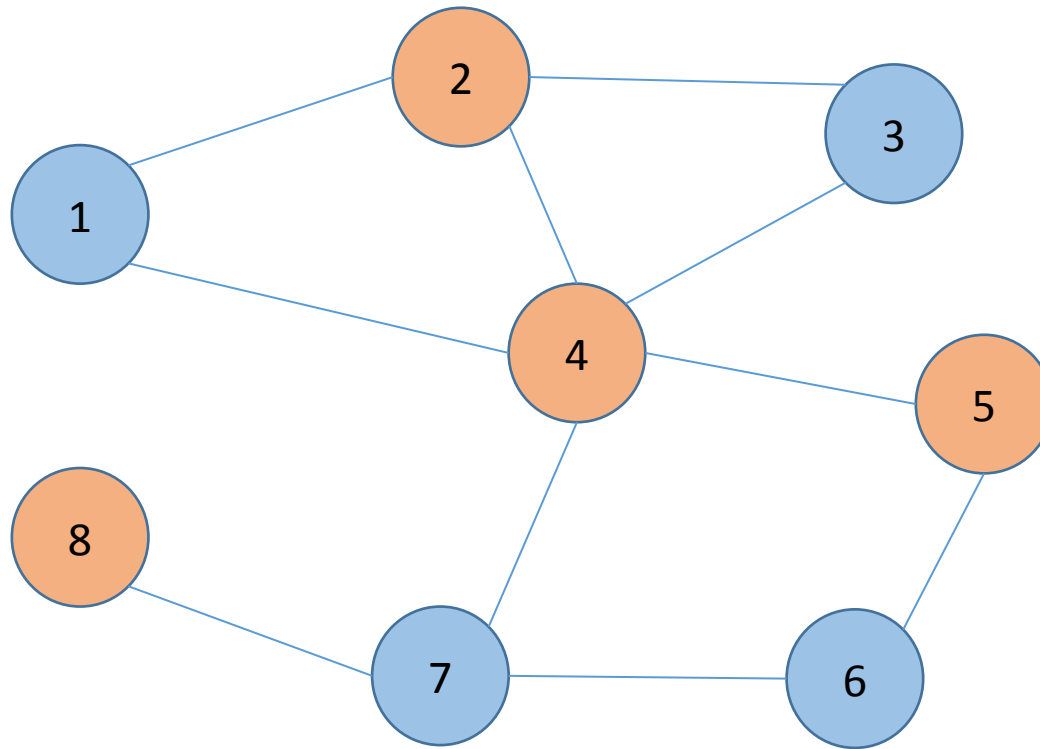
$A = \{1, 3, 6, 7\}$

$B = \{2, 4, 5, 8\}$



Couldn't we just use graph
coloring for making sets?

Consider the Following Graph

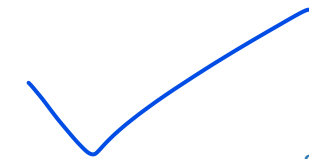


Let us group the vertices into two sets

$A = \{1, 3, 6, 7\}$

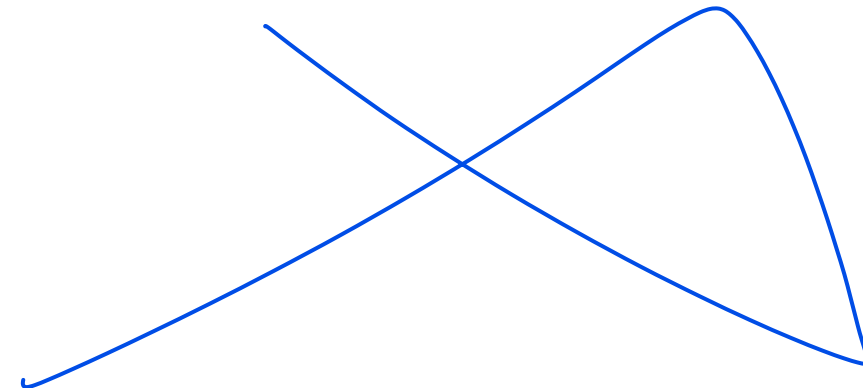
$B = \{2, 4, 5, 8\}$

Couldn't we just use graph coloring for making sets?

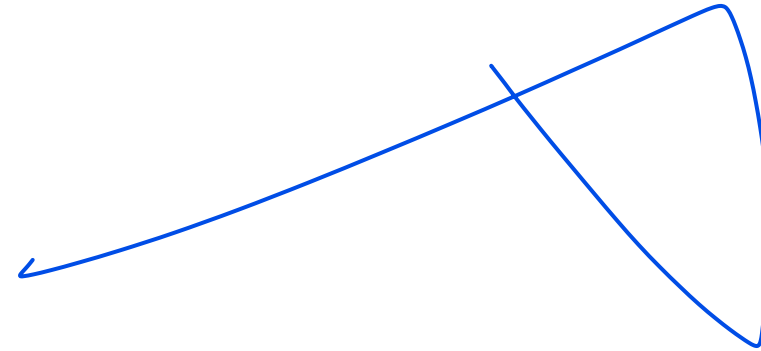
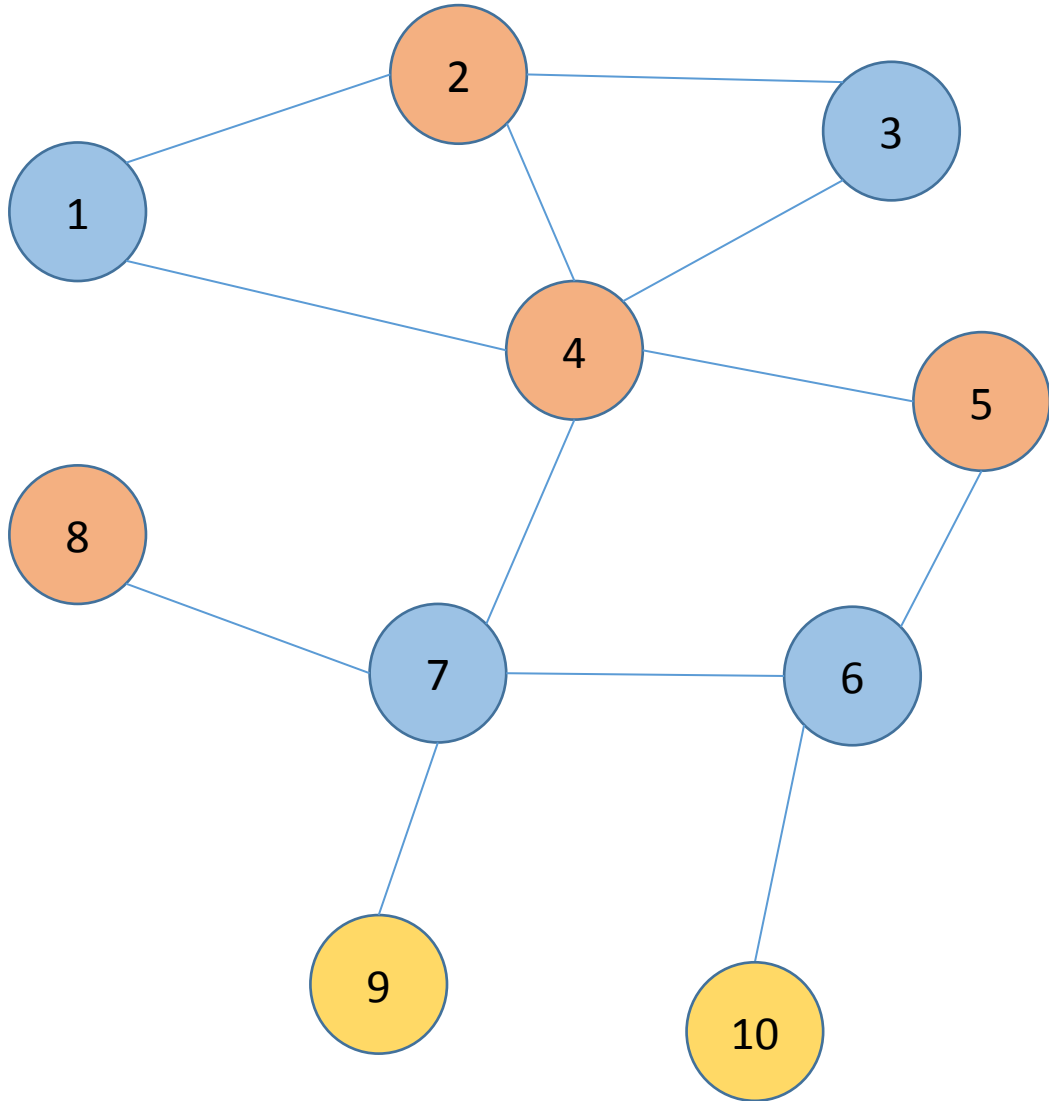


Set Representation

- Couldn't we just use graph coloring for making sets?
 - No
 - There are different set operations
 - **Union** is one important set operation
 - Hence we need a different arrangement
 - Also sets are not used in other cases not related to graphs

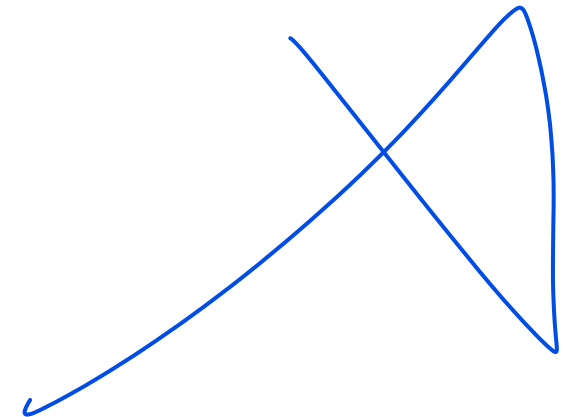
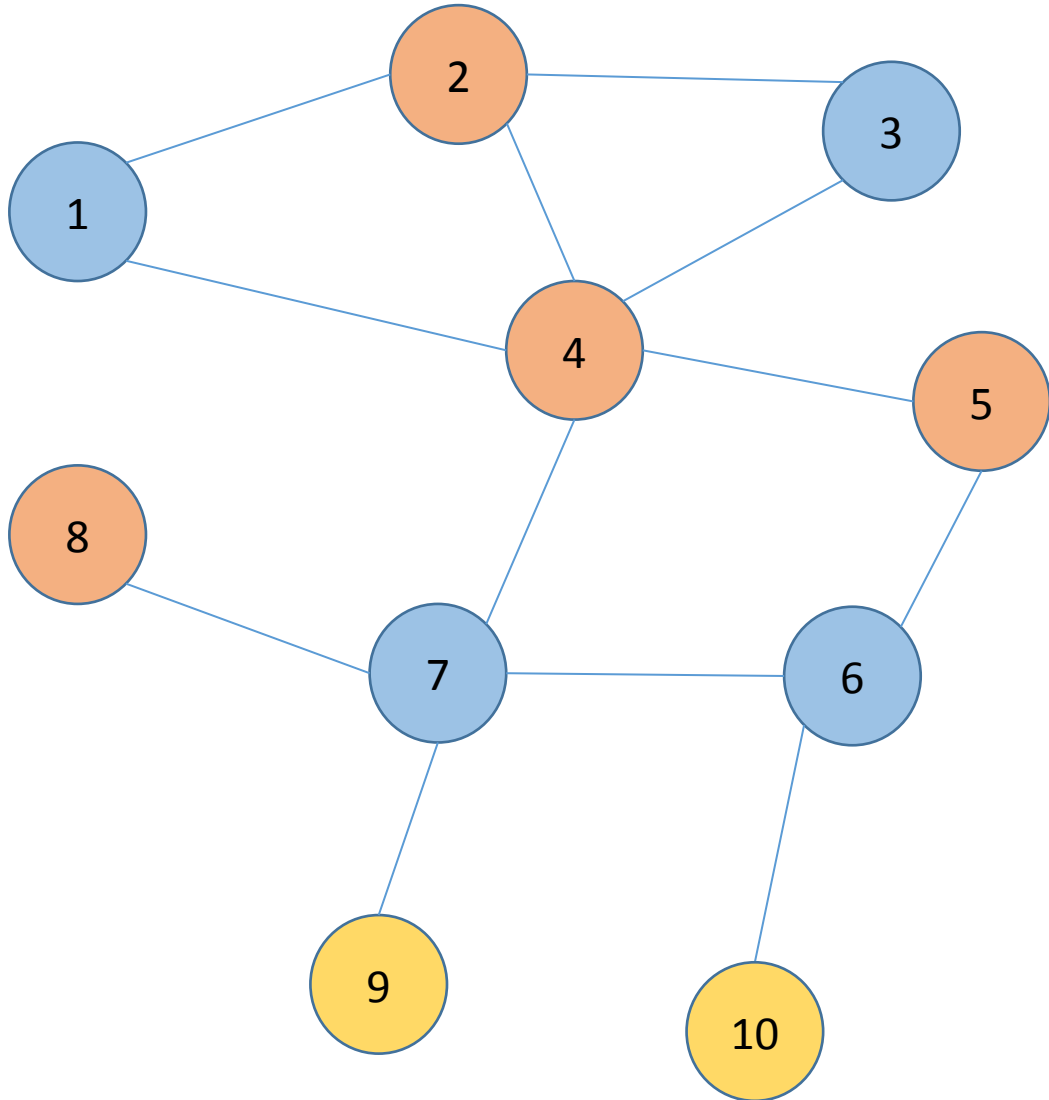


Set Representation

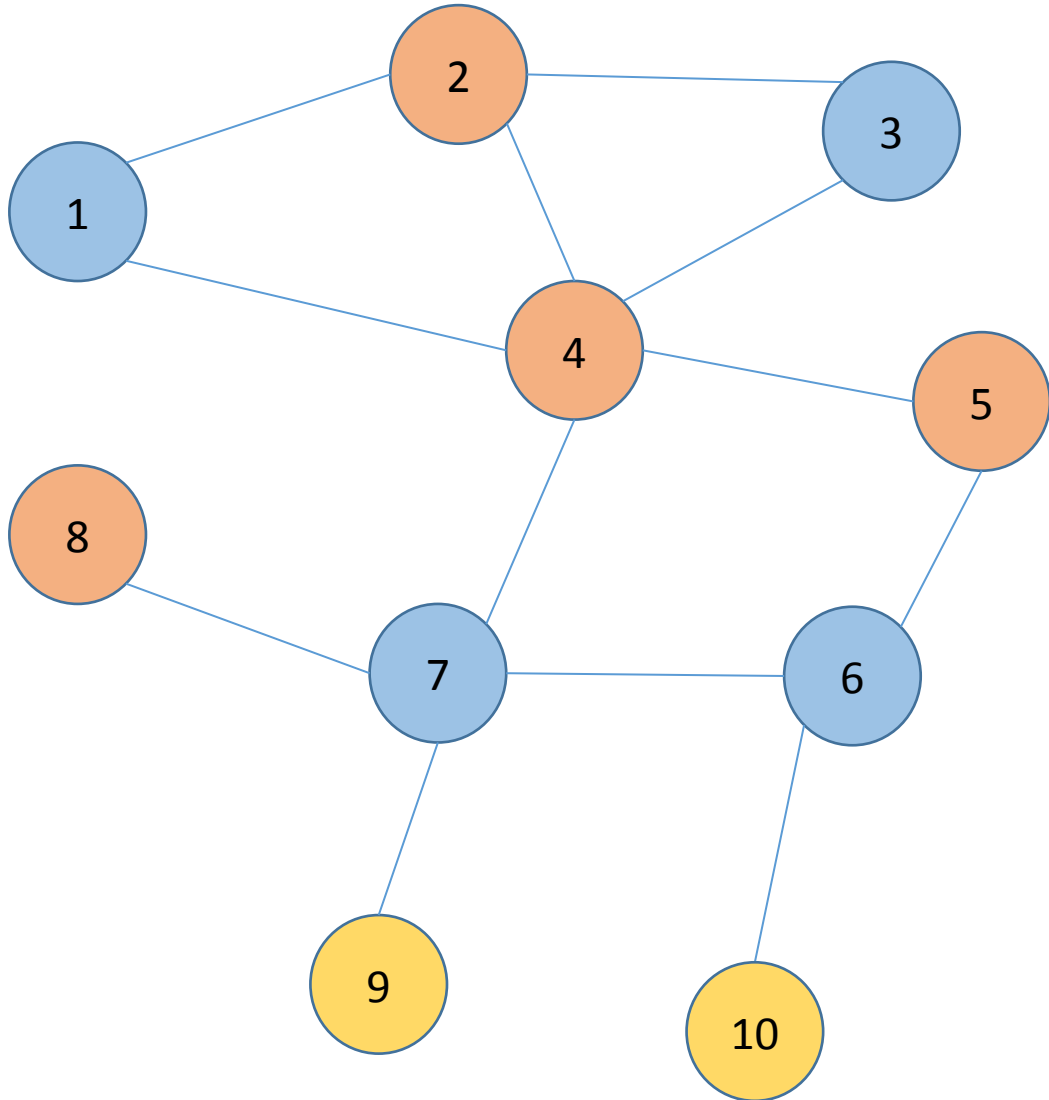


Set Representation

Array and Chaining:

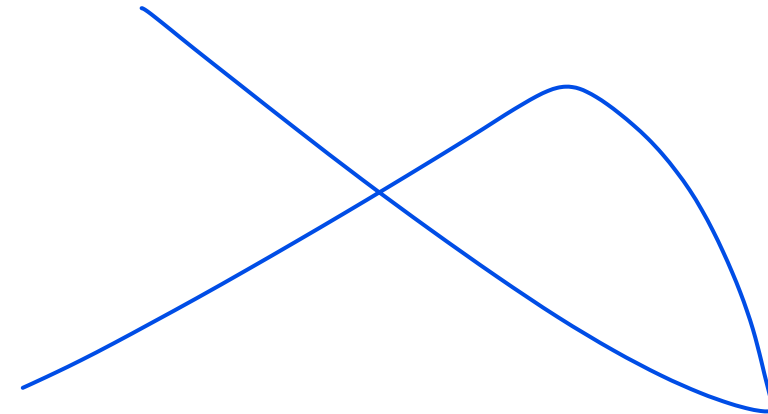


Set Representation

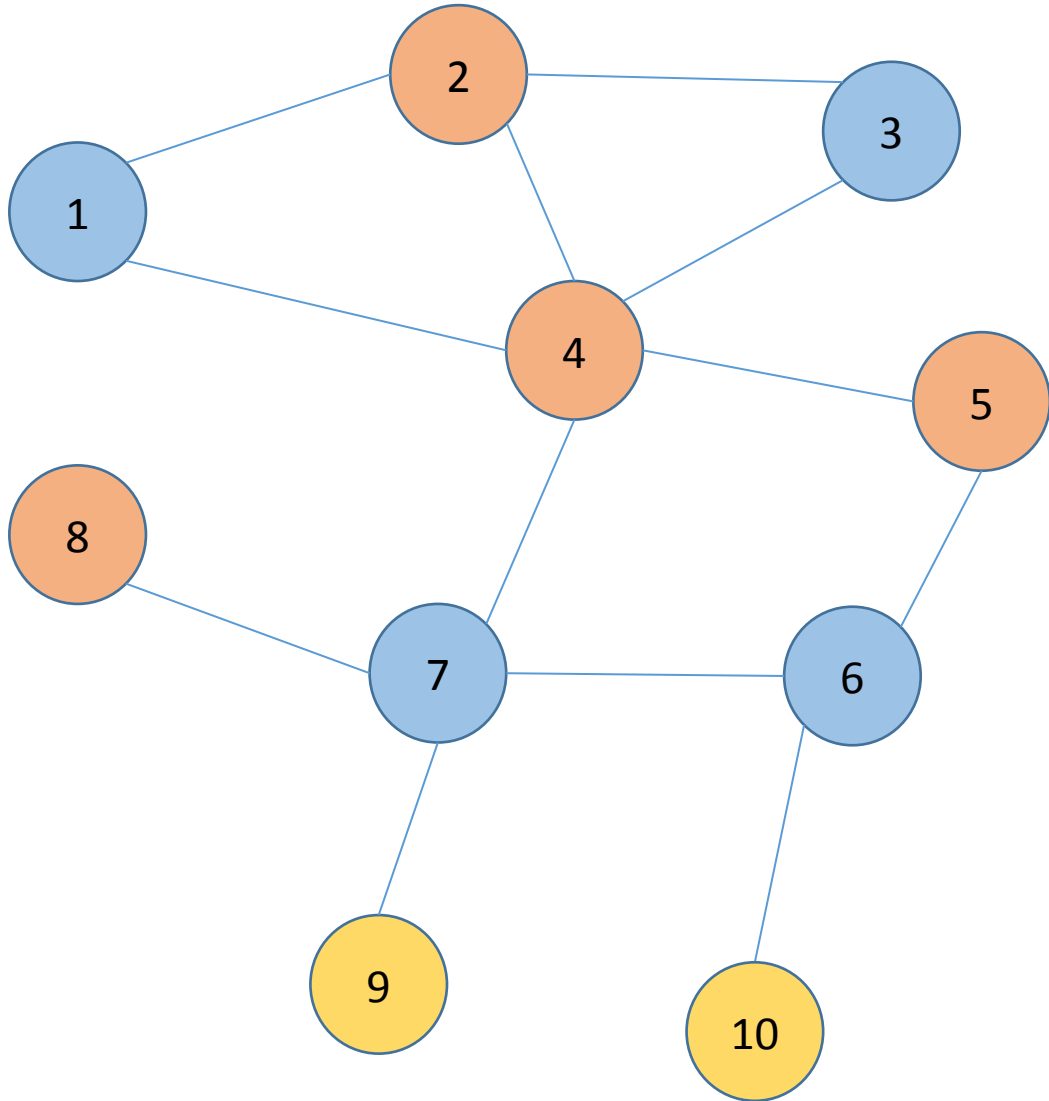


Array and Chaining:

1	2	3	4	5	6	7	8	9	10
1	4	1	4	4	7	3	2	9	9

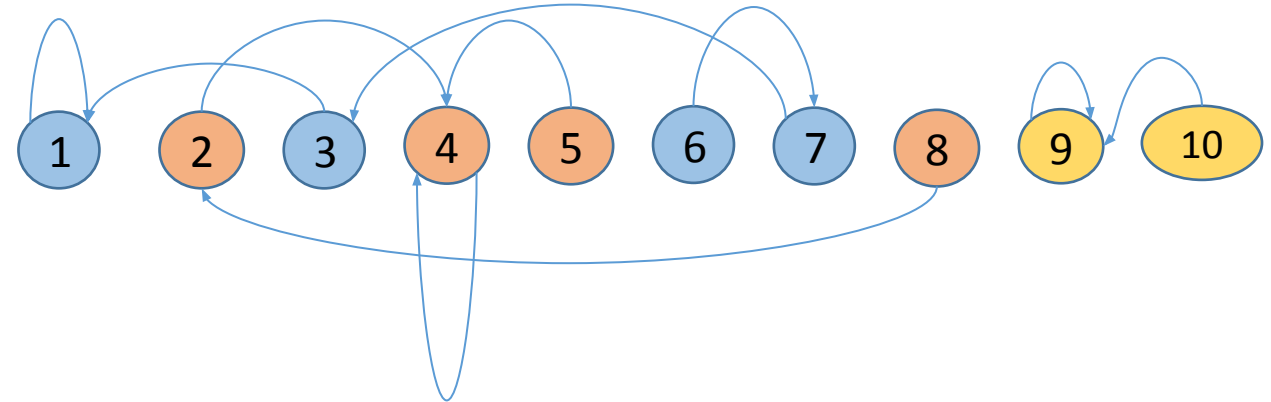


Set Representation

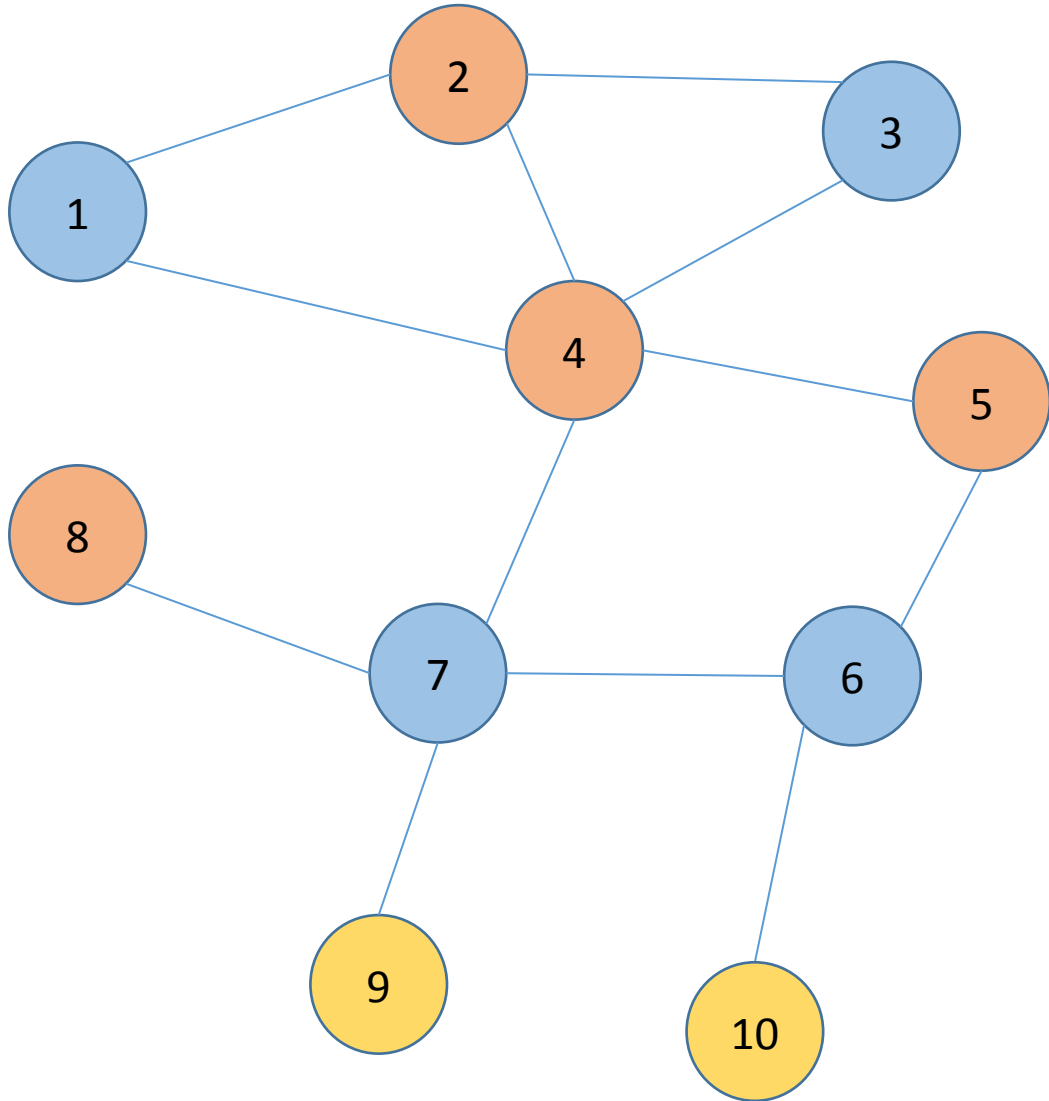


Array and Chaining:

1	2	3	4	5	6	7	8	9	10
1	4	1	4	4	7	3	2	9	9

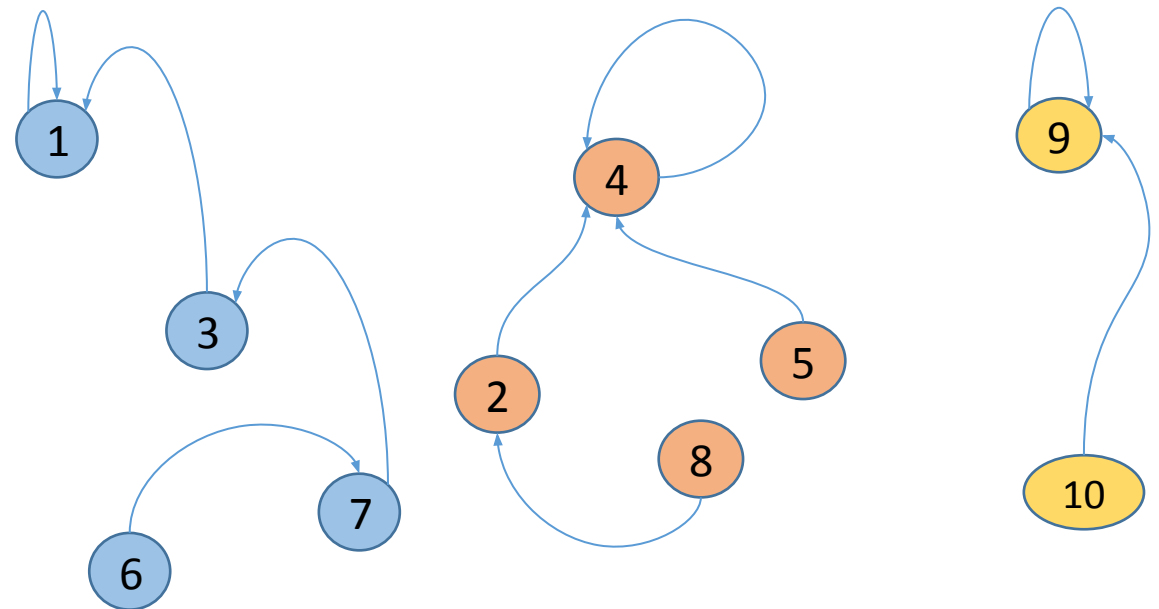


Set Representation

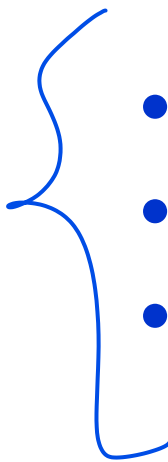


Array and Chaining:

1	2	3	4	5	6	7	8	9	10
1	4	1	4	4	7	3	2	9	9



Set Functions

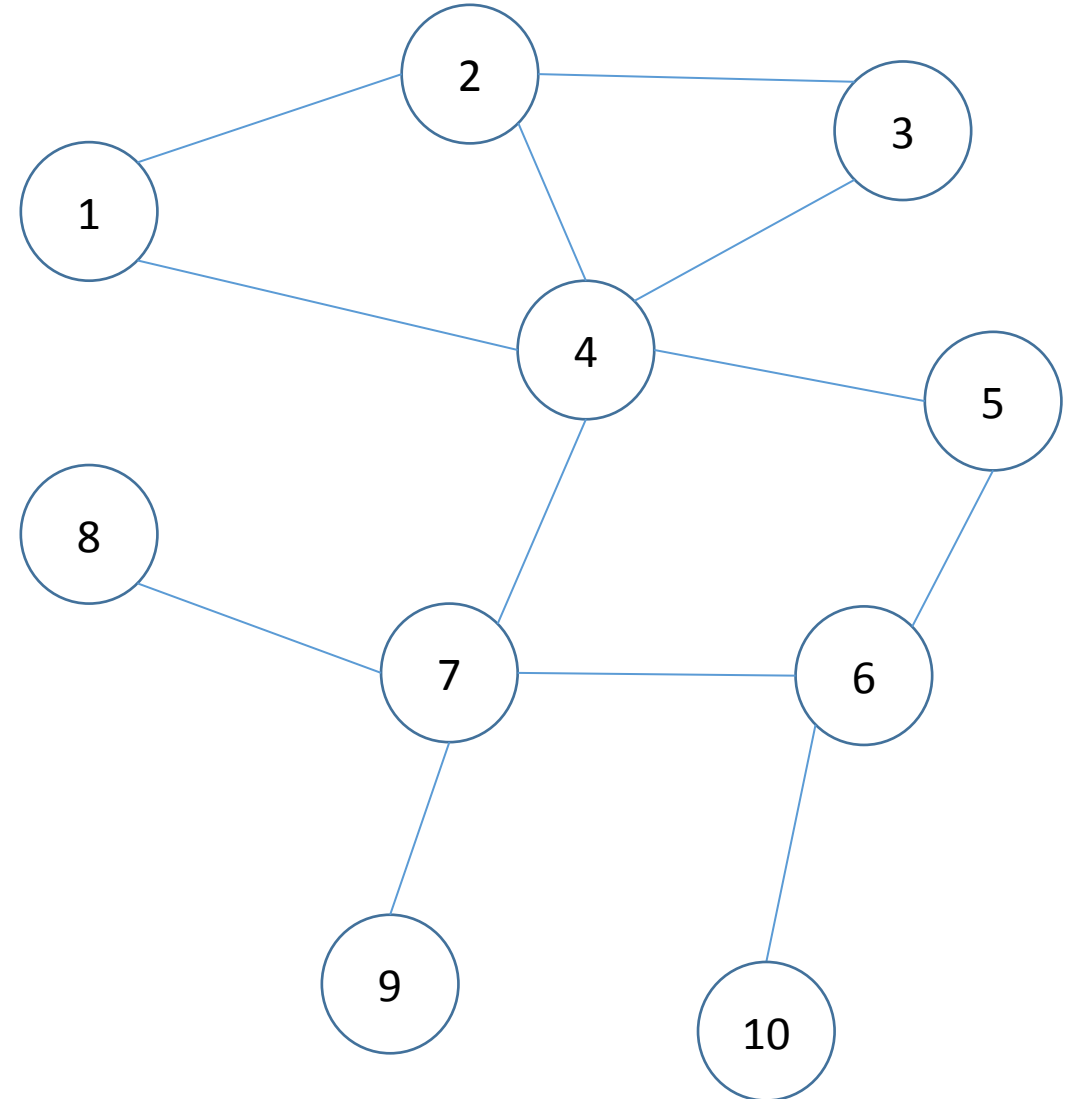
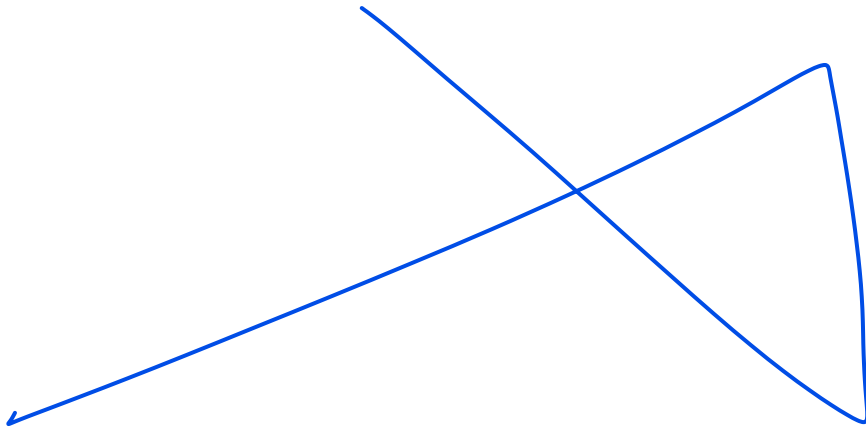
- 
- MakeSet()
 - FindSet()/FindParent()
 - Union()

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

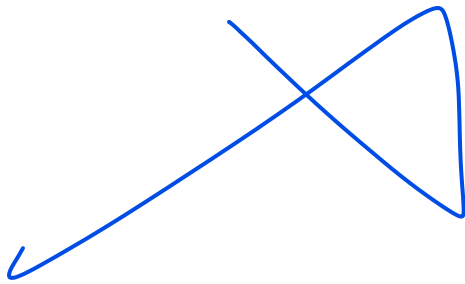
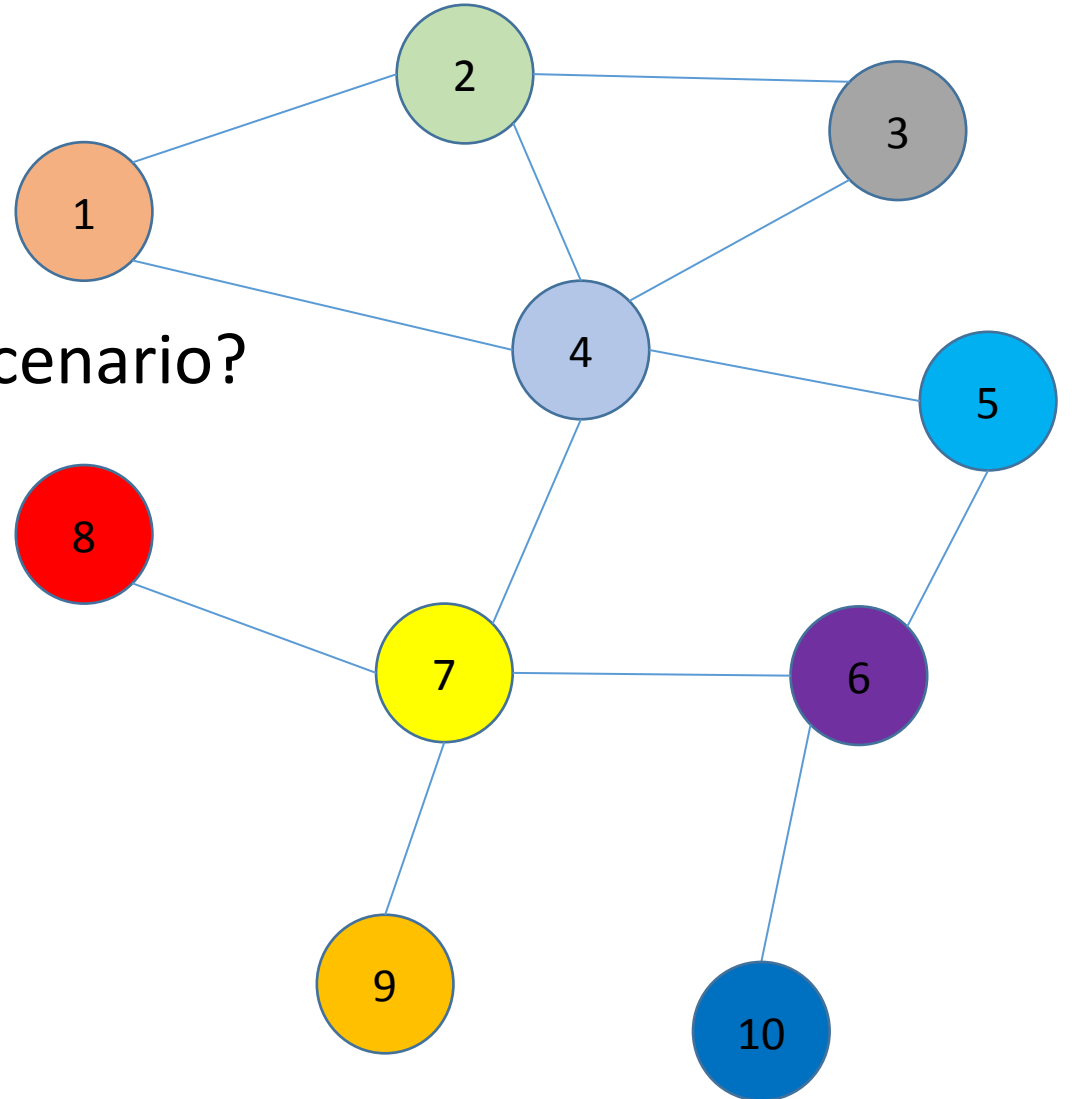
Set Functions

- MakeSet()
 - Consider each element an individual set



Set Functions

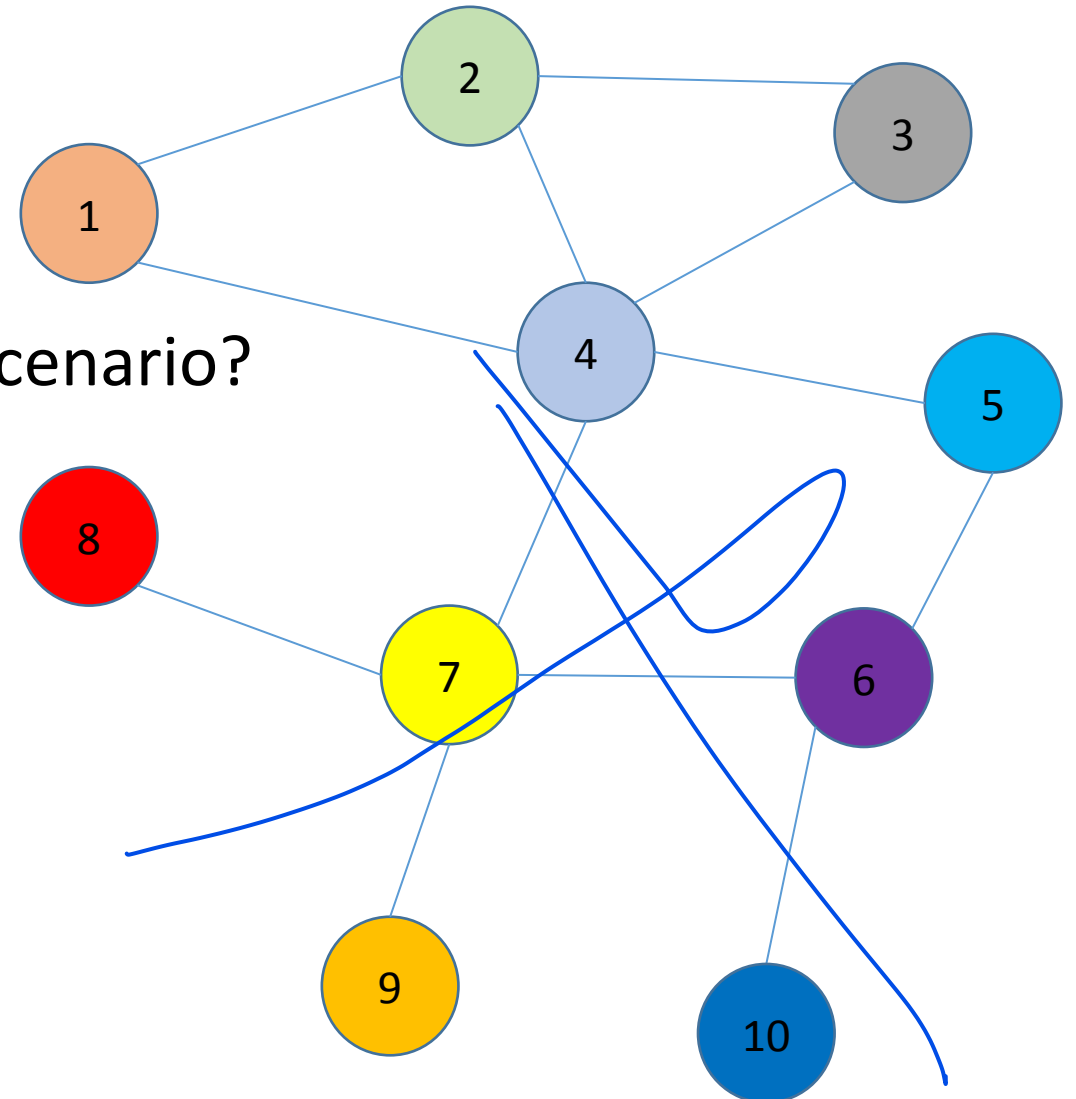
- MakeSet()
 - Consider each element an individual set
- How are you going to represent this scenario?



Set Functions

- MakeSet()
 - Consider each element an individual set
- How are you going to represent this scenario?

1	2	3	4	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10

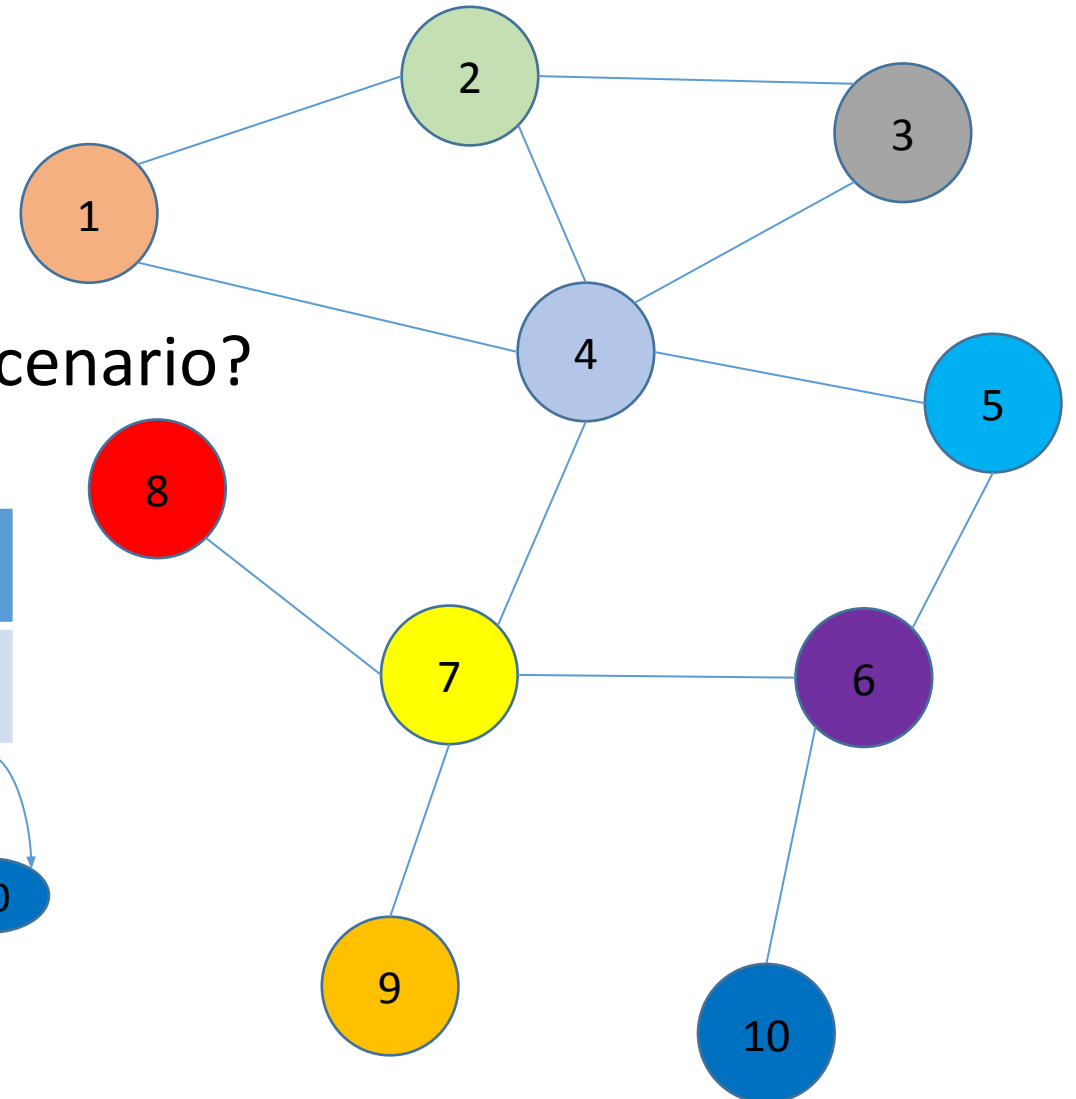
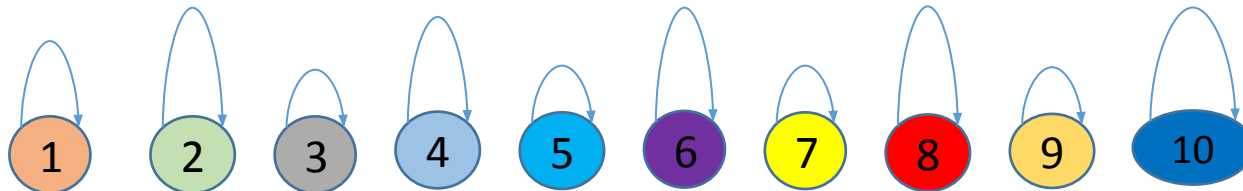


Set Functions

- MakeSet()
 - Consider each element an individual set

- How are you going to represent this scenario?

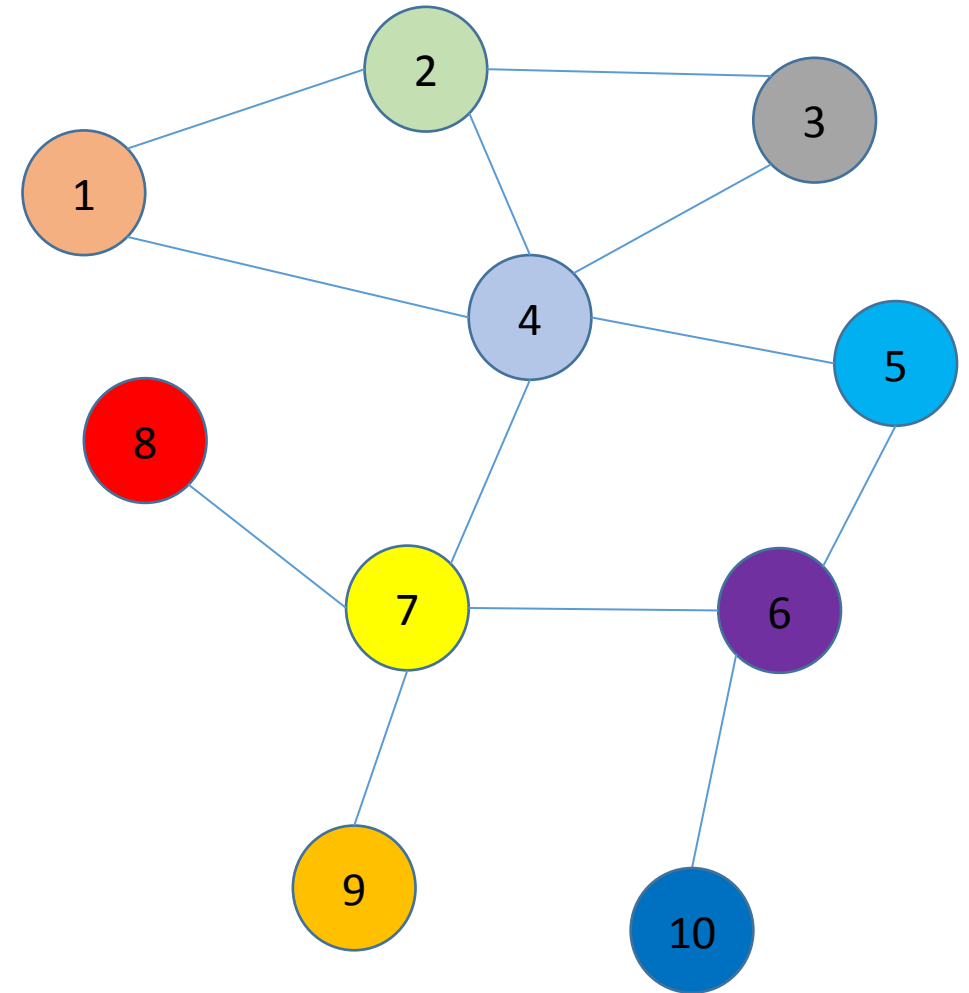
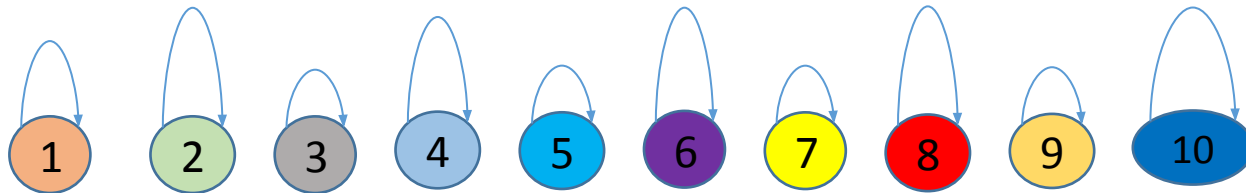
1	2	3	4	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10



Set Functions

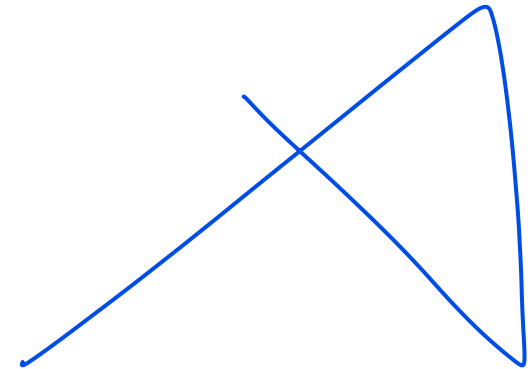
```
void makeSet() {  
    for i = 1 to n  
        parent[i] = i  
}
```

1	2	3	4	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10



Set Functions

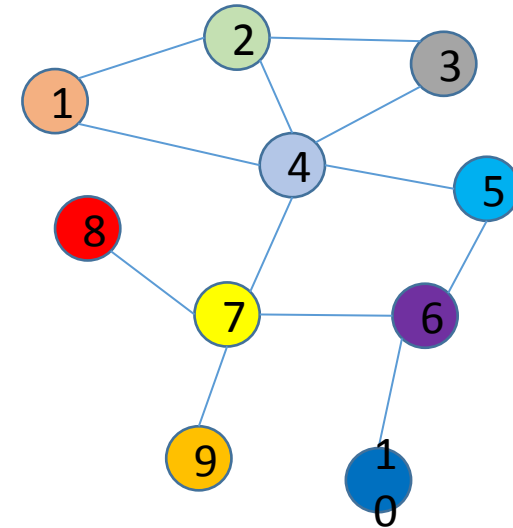
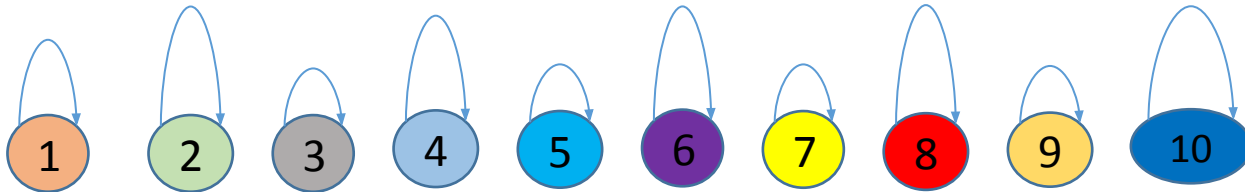
- Union(a,b)
 - Brings two elements under the same set
 - In the given case for graphs – brings two vertices under the same parent



Set Functions

- Union(a,b)
 - Brings two elements under the same set
 - In the given case for graphs – brings two vertices under the same parent

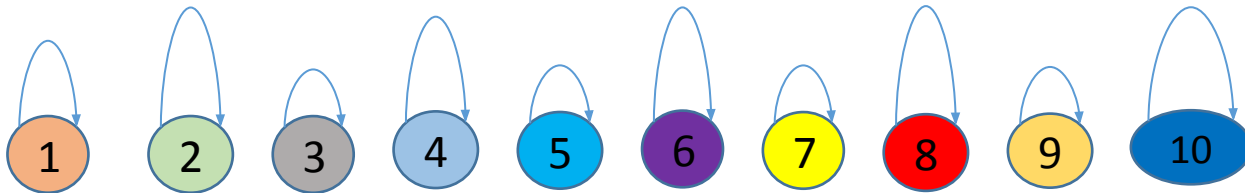
1	2	3	4	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10



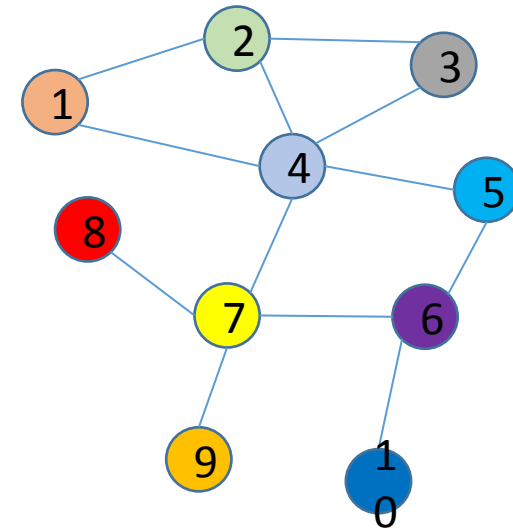
Set Functions

- Union(a,b)
 - Brings two elements under the same set
 - In the given case for graphs – brings two vertices under the same parent

1	2	3	4	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10



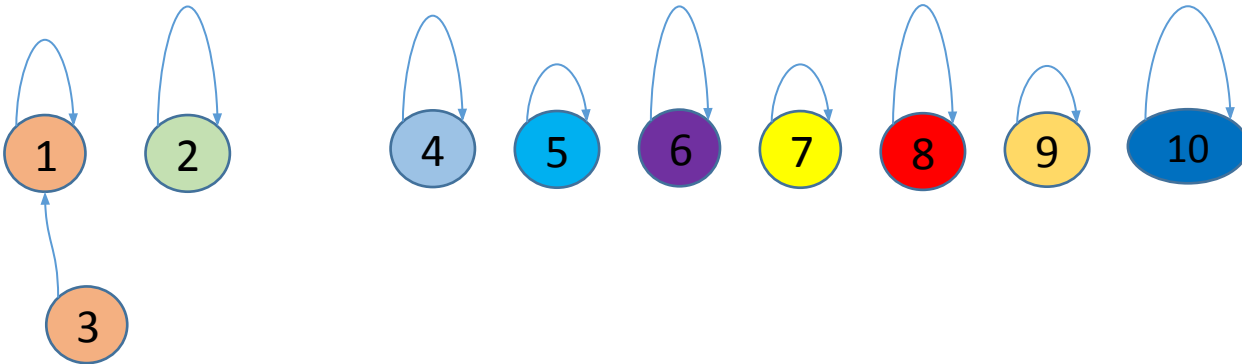
Union(1,3)



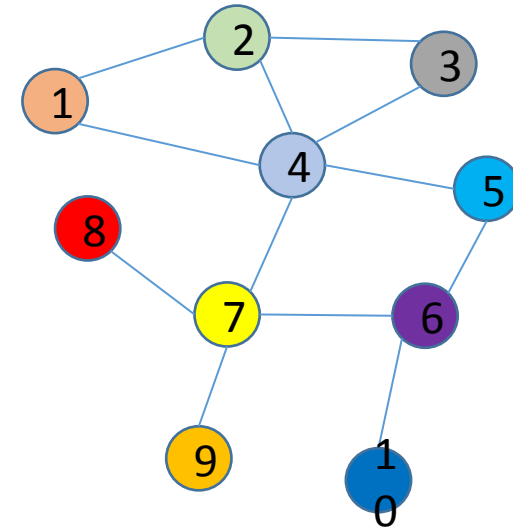
Set Functions

- Union(a,b)
 - Brings two elements under the same set
 - In the given case for graphs – brings two vertices under the same parent

1	2	3	4	5	6	7	8	9	10
1	2	1	4	5	6	7	8	9	10



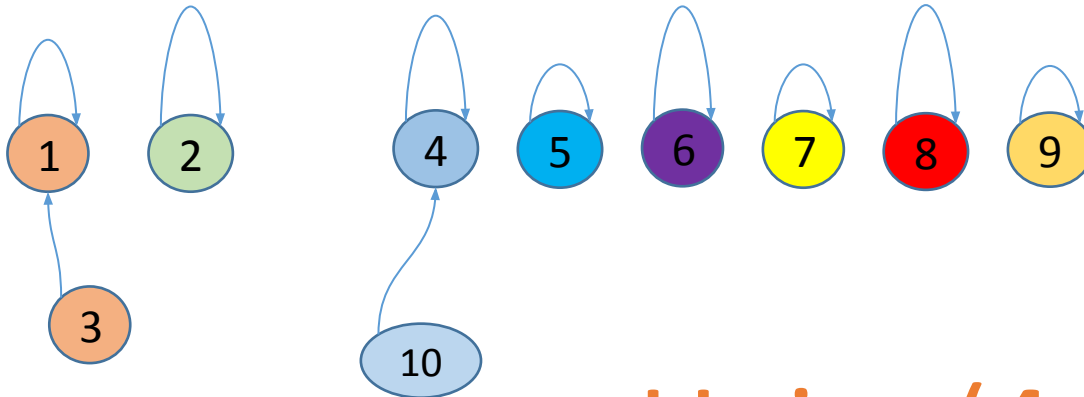
Union(1,3)



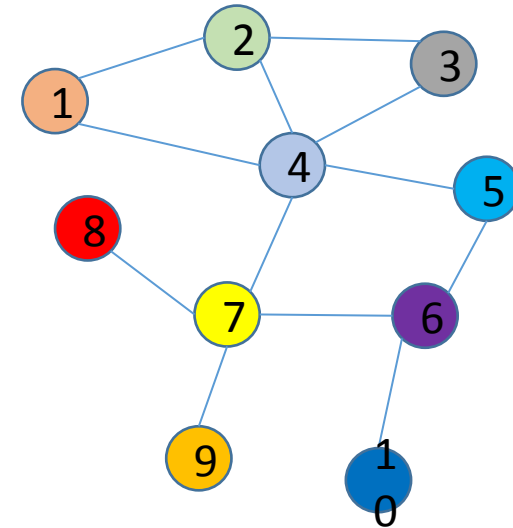
Set Functions

- Union(a,b)
 - Brings two elements under the same set
 - In the given case for graphs – brings two vertices under the same parent

1	2	3	4	5	6	7	8	9	10
1	2	1	4	5	6	7	8	9	4



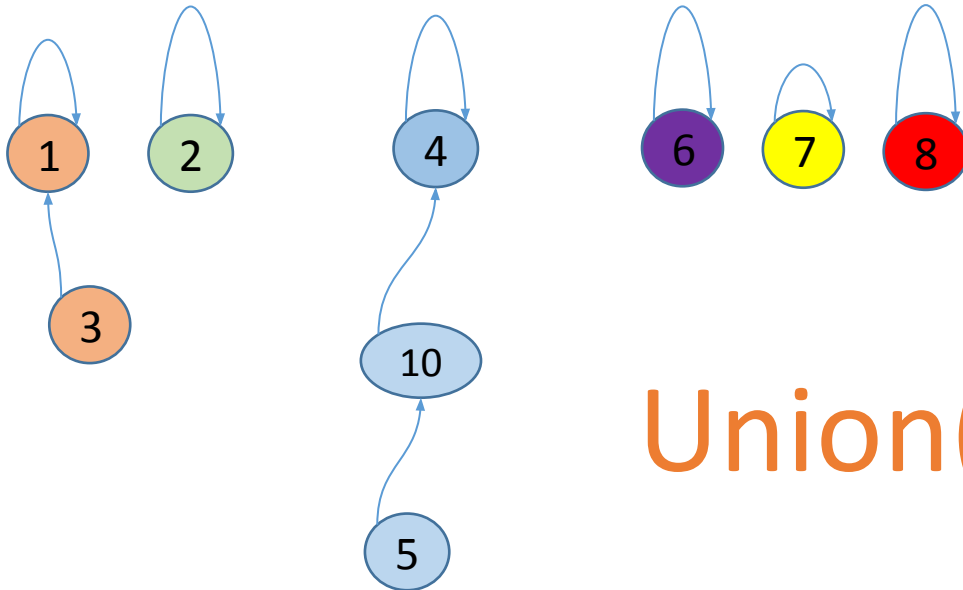
Union(4,10)



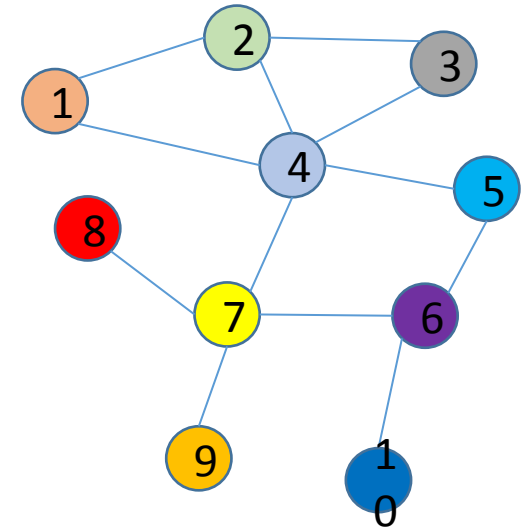
Set Functions

- Union(a,b)
 - Brings two elements under the same set
 - In the given case for graphs – brings two vertices under the same parent

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	6	7	8	9	10



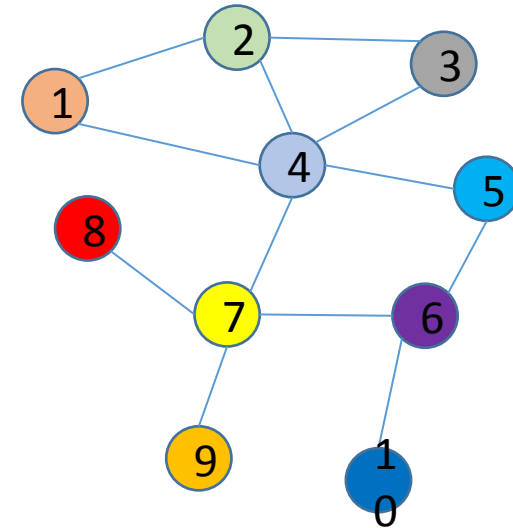
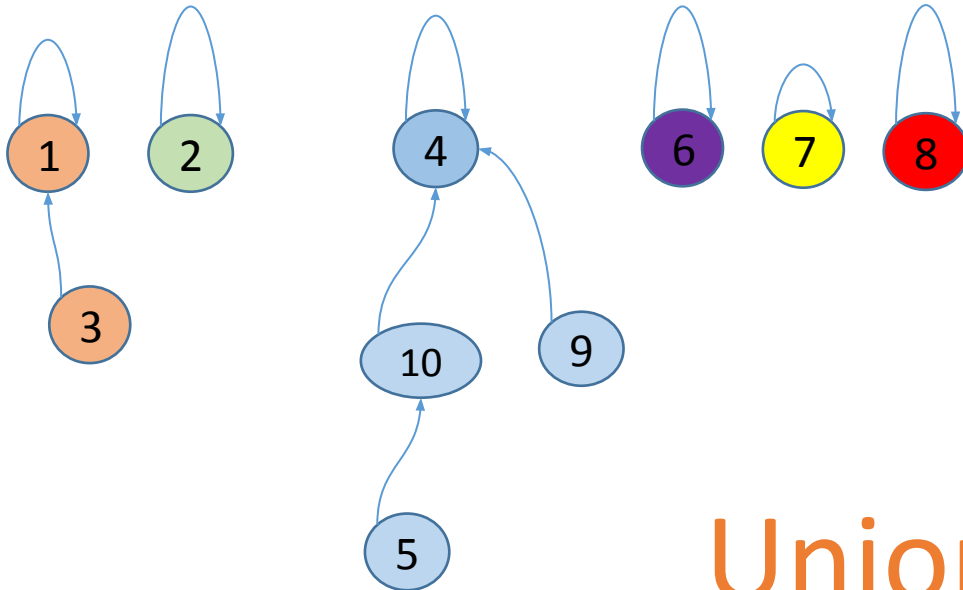
Union(10,5)



Set Functions

- Union(a,b)
 - Brings two elements under the same set
 - In the given case for graphs – brings two vertices under the same parent

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	6	7	8	9	10

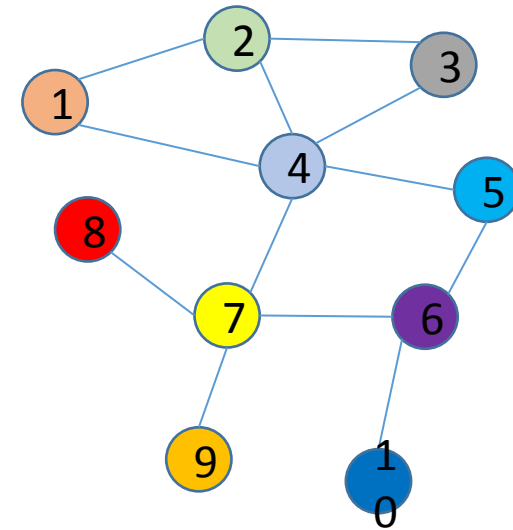
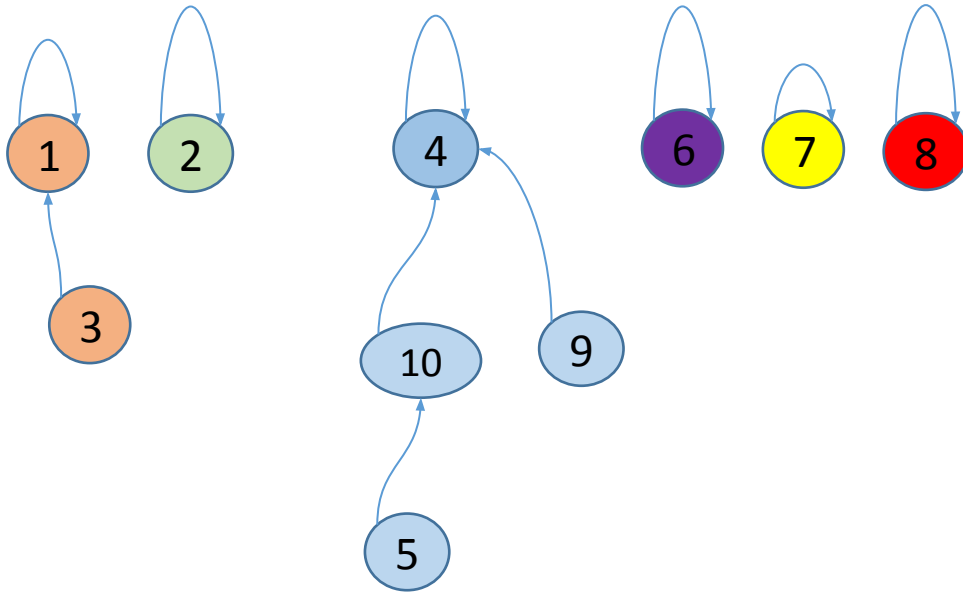


Union(4,9)

Set Functions

- Union(a,b)
 - Brings two elements under the same set
 - In the given case for graphs – brings two vertices under the same parent

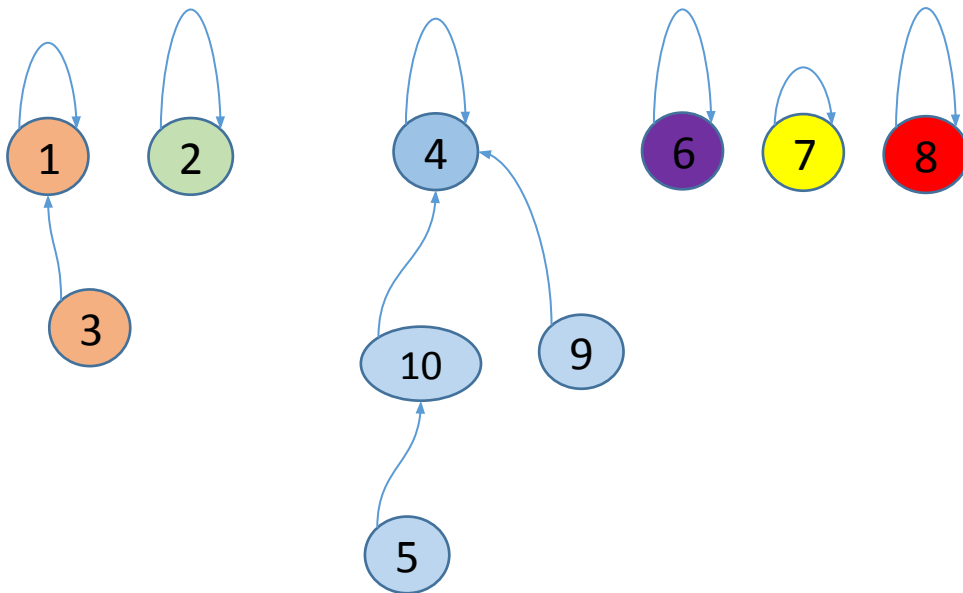
1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	6	7	8	9	10



Set Functions

- Union(a,b)
 - Brings two elements under the same set
 - In the given case for graphs – brings two vertices under the same parent

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	6	7	8	9	10

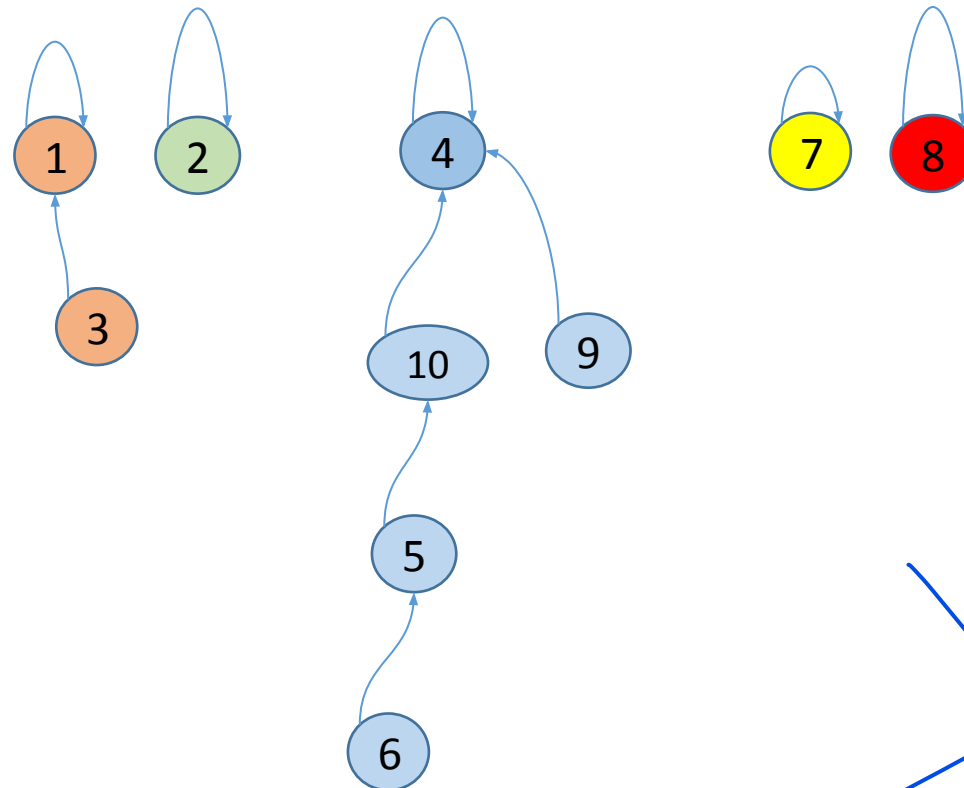


```
Union (a,b)
  u = findParent(a)
  v = findParent(b)
  if u==v
    print("From same set")
  else
    parent[b]=a
```

Set Functions

- FindParent(a)
 - Returns the root.

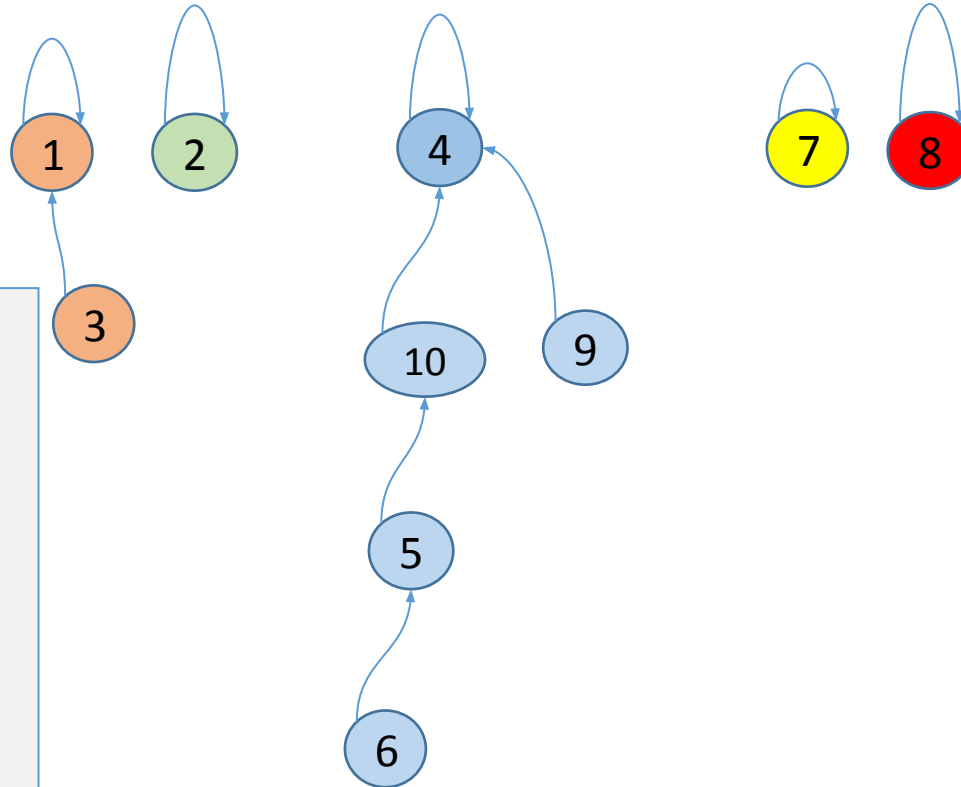
1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10

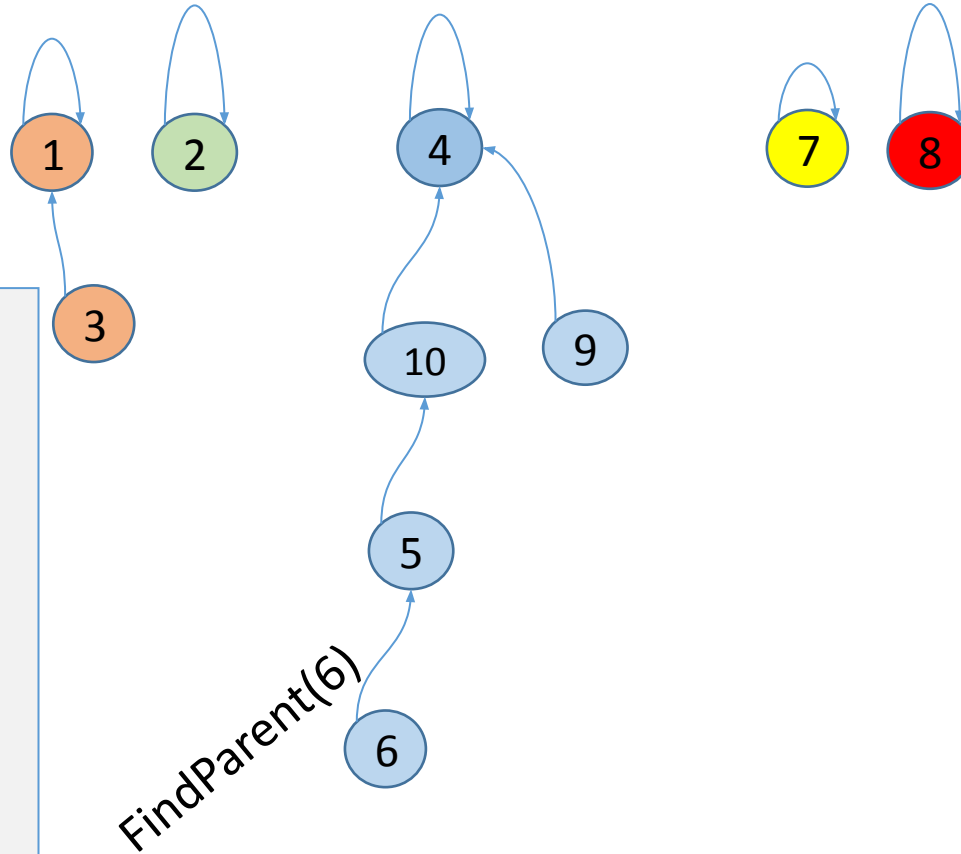


```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10

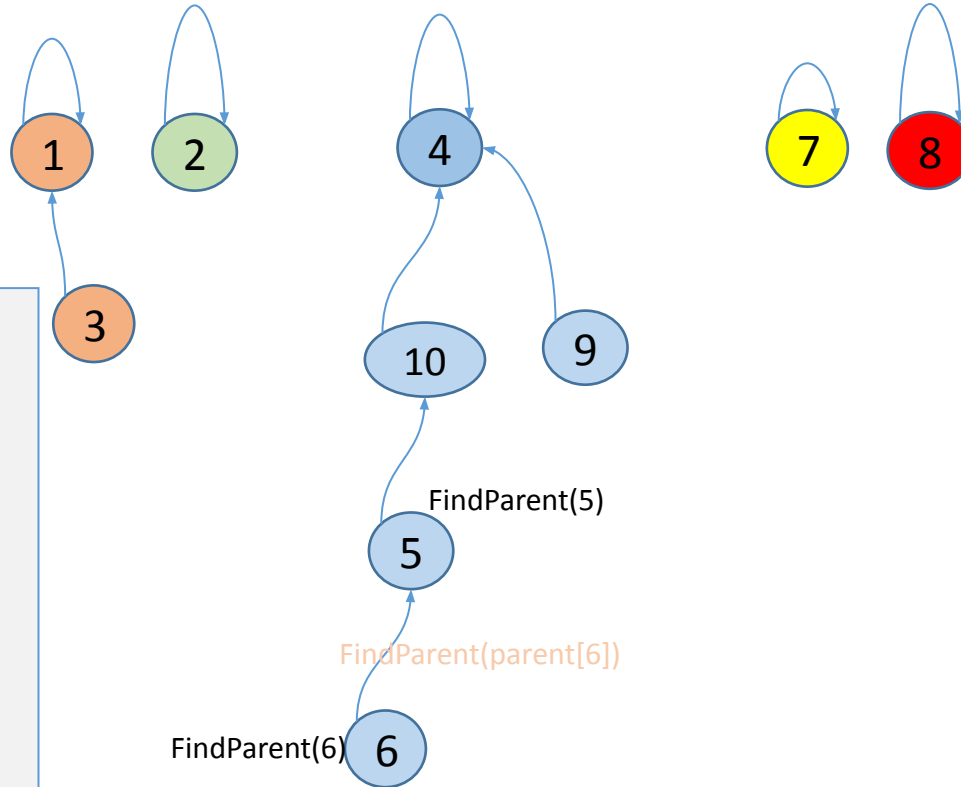


```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10

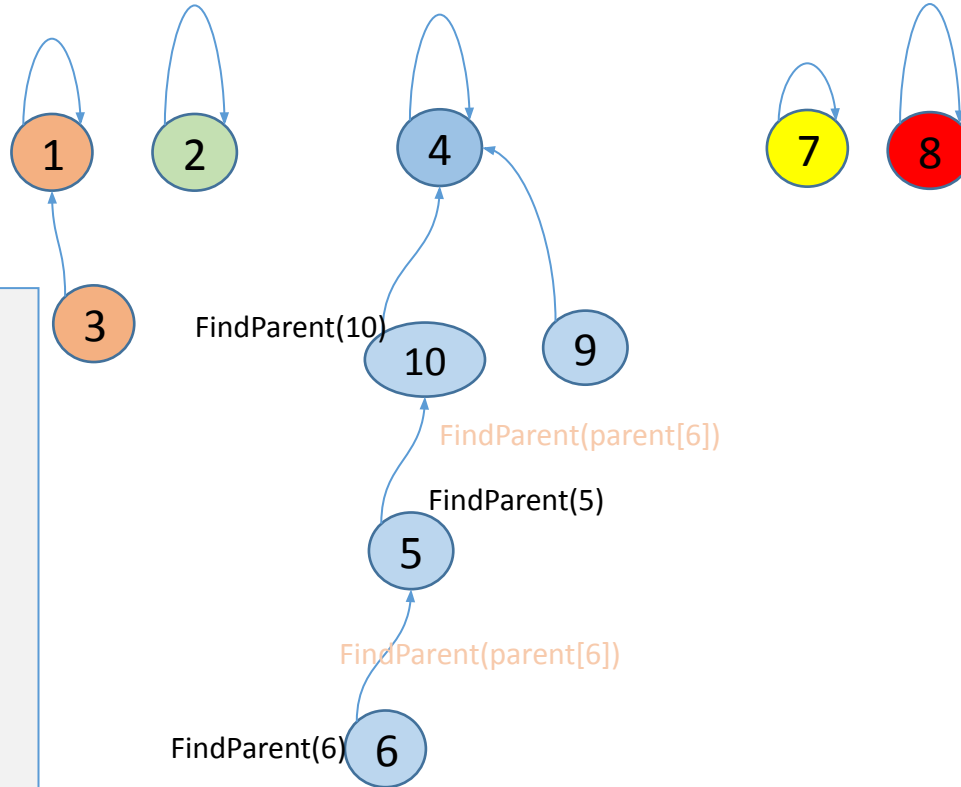


```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10

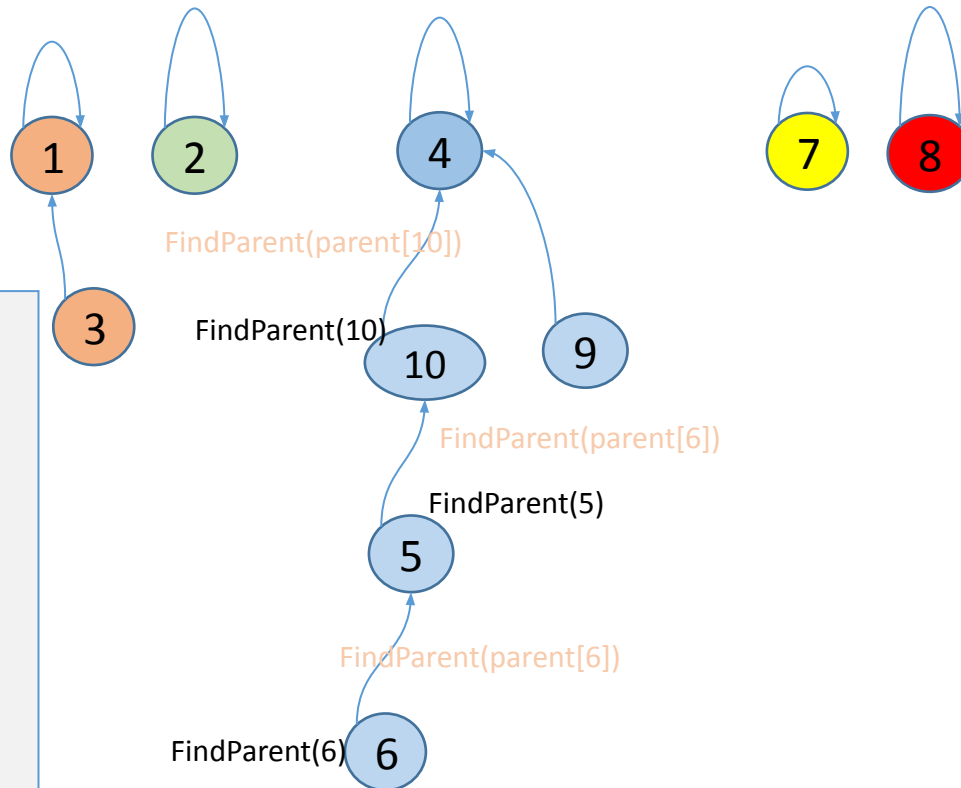


```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10

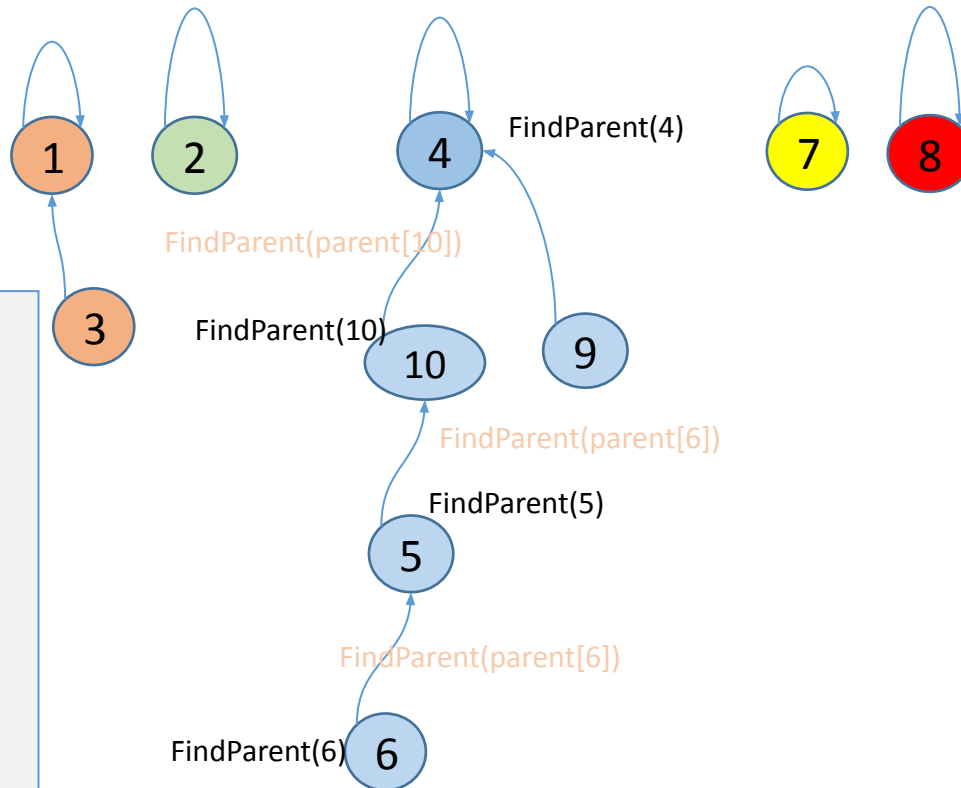


```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10

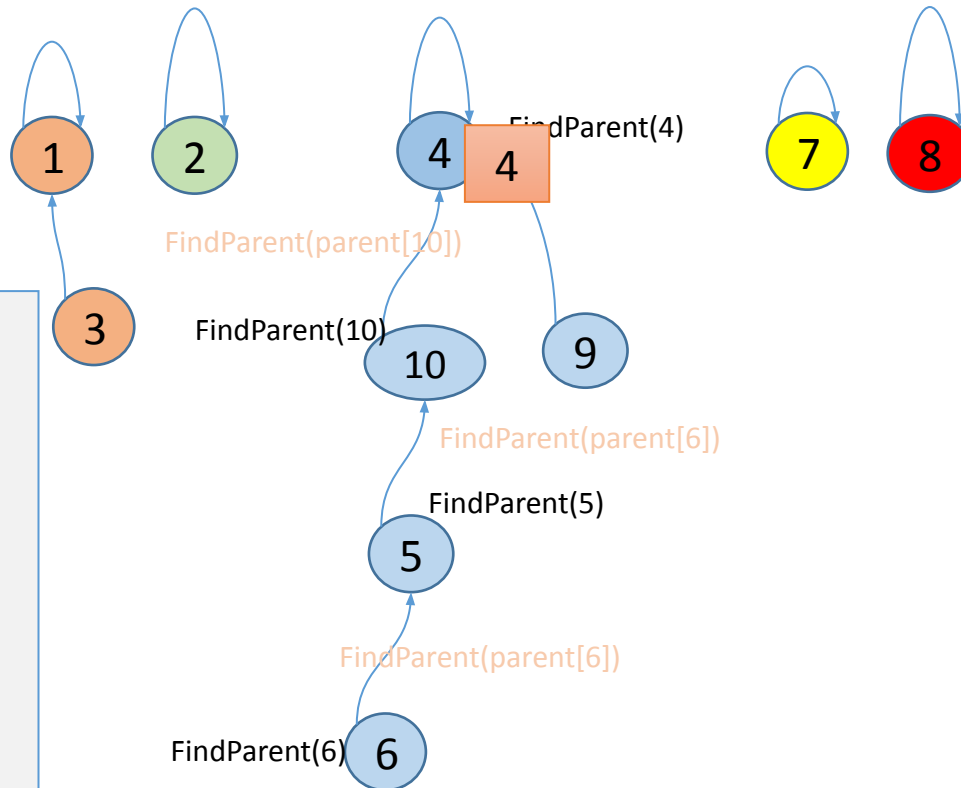


```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10

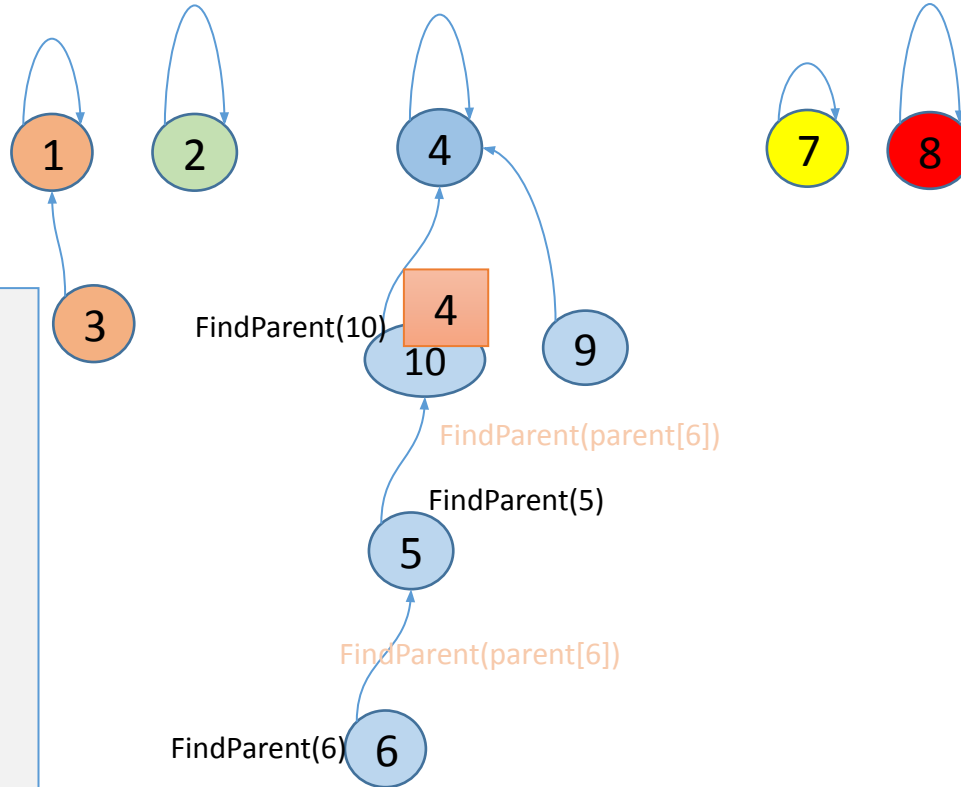


```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10

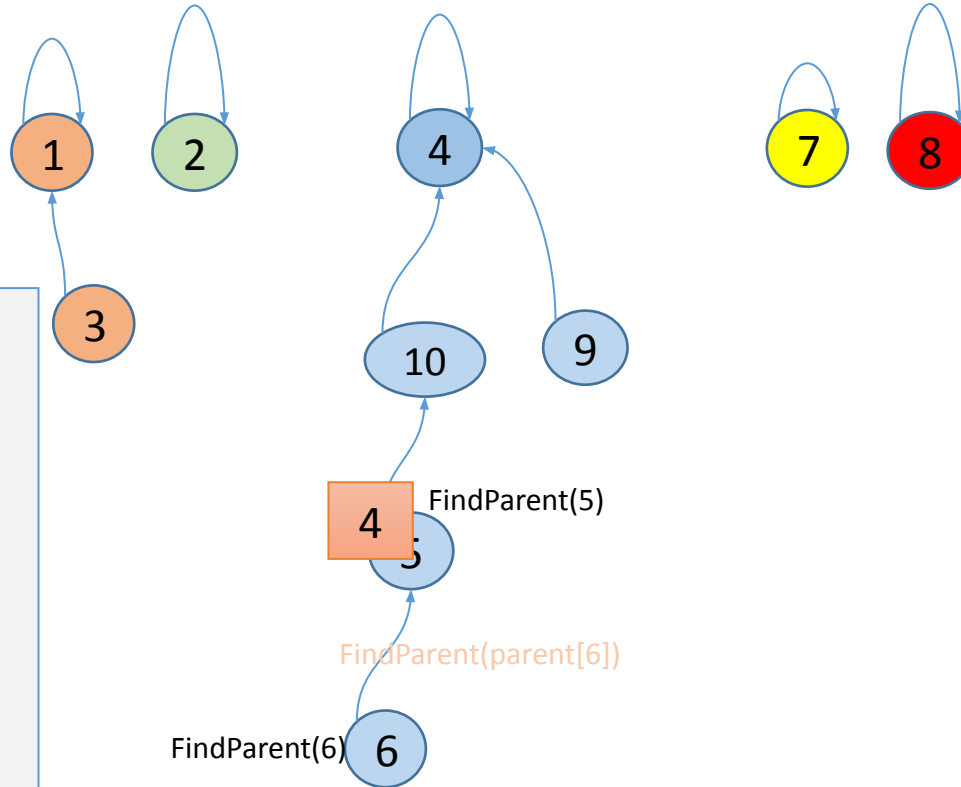


```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10

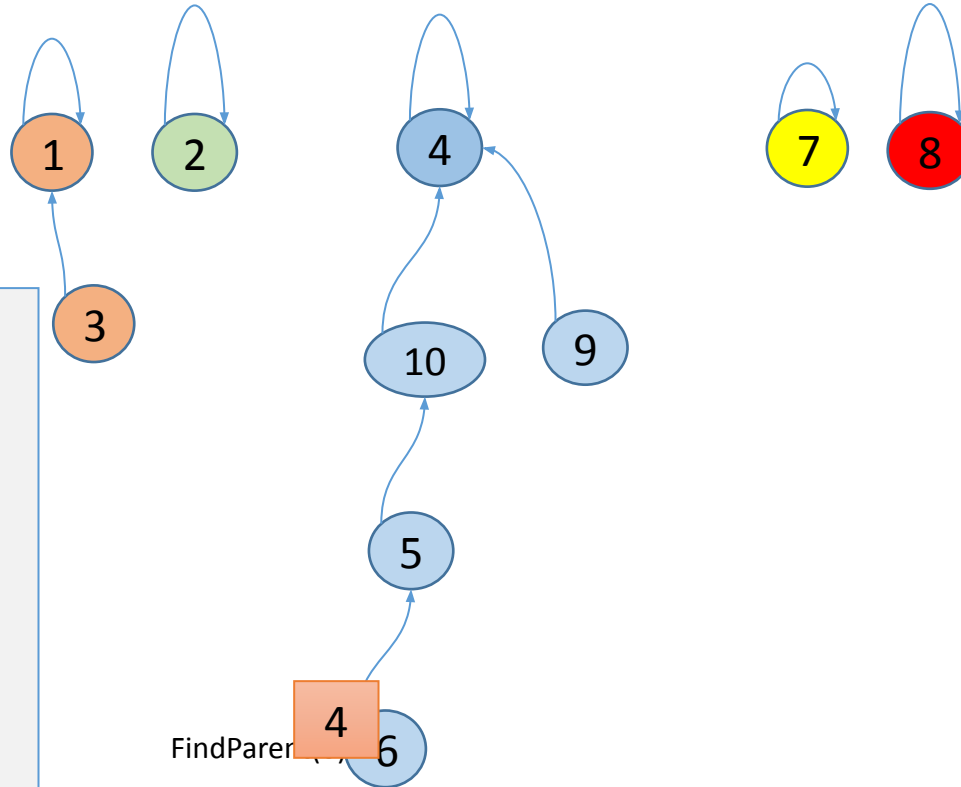


```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10

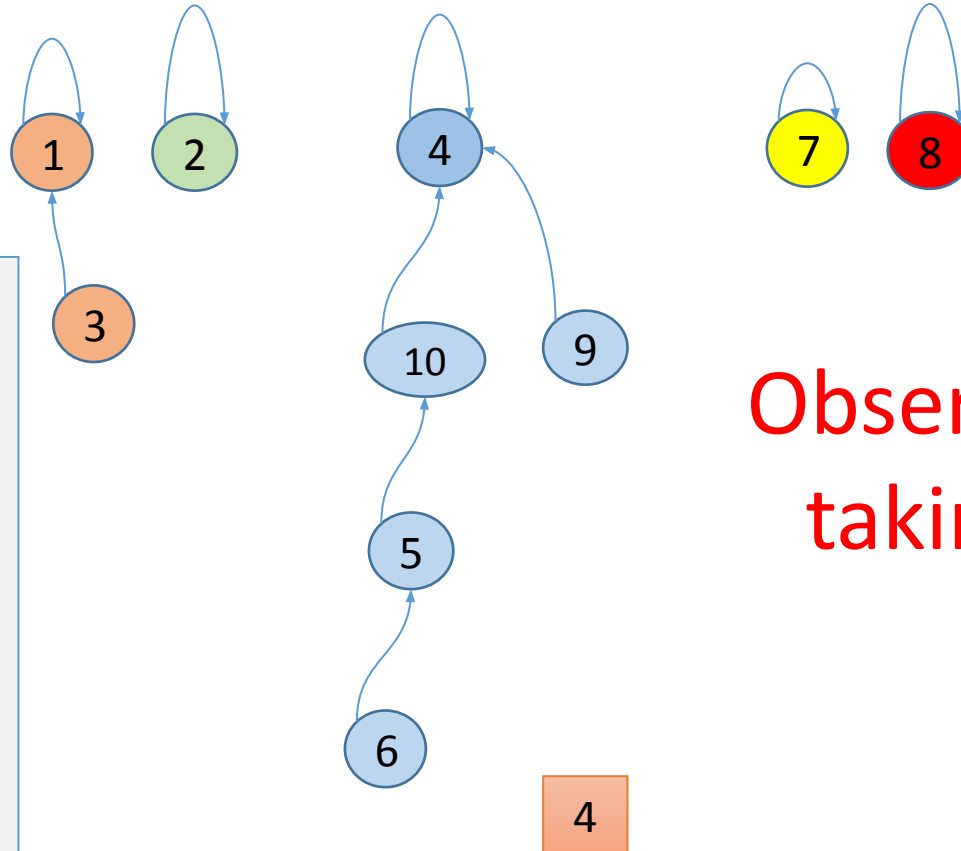


```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



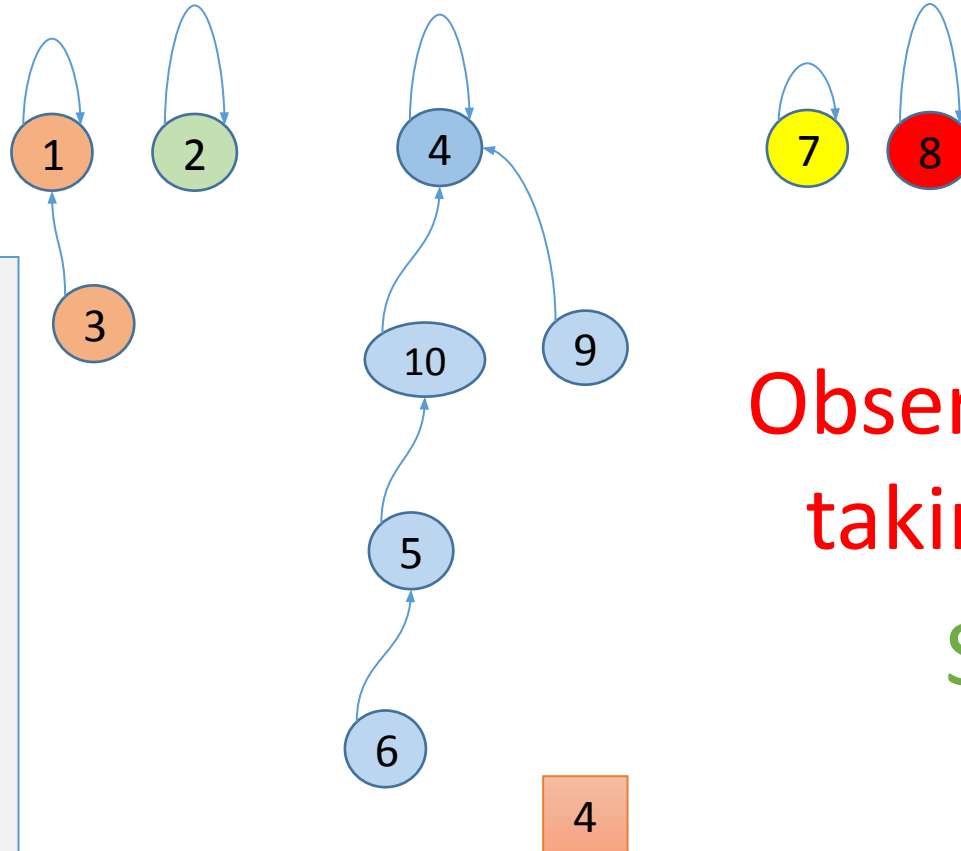
```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Observe that this is
taking $O(n)$ time

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



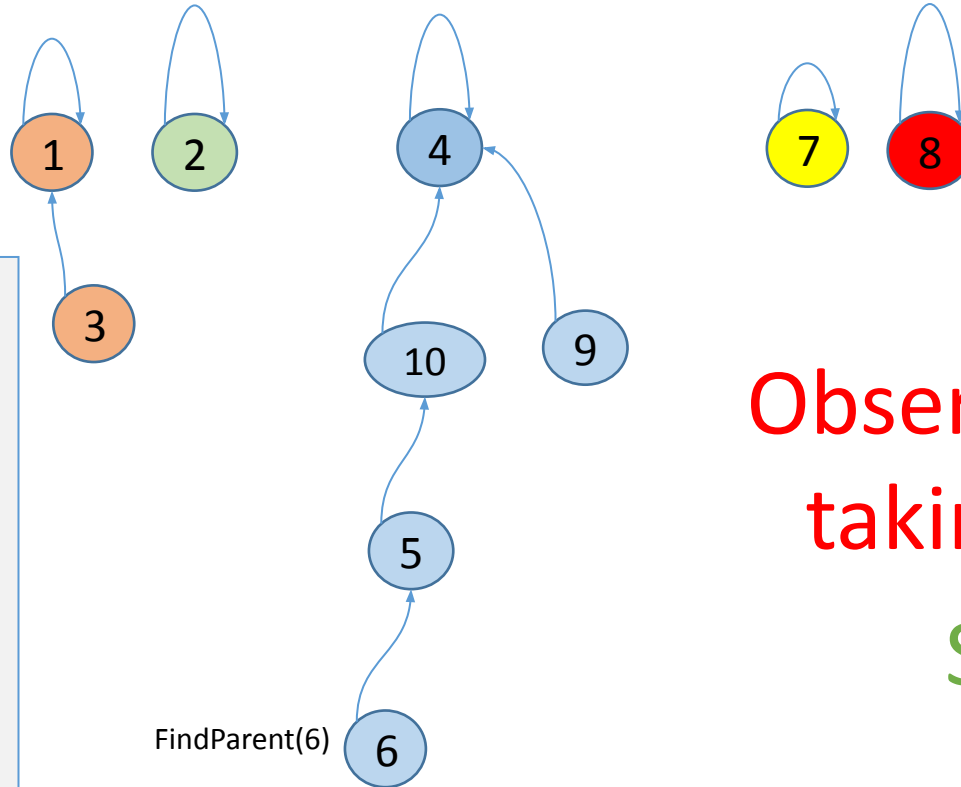
```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
return x
```

Observe that this is
taking $O(n)$ time
Solution?

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



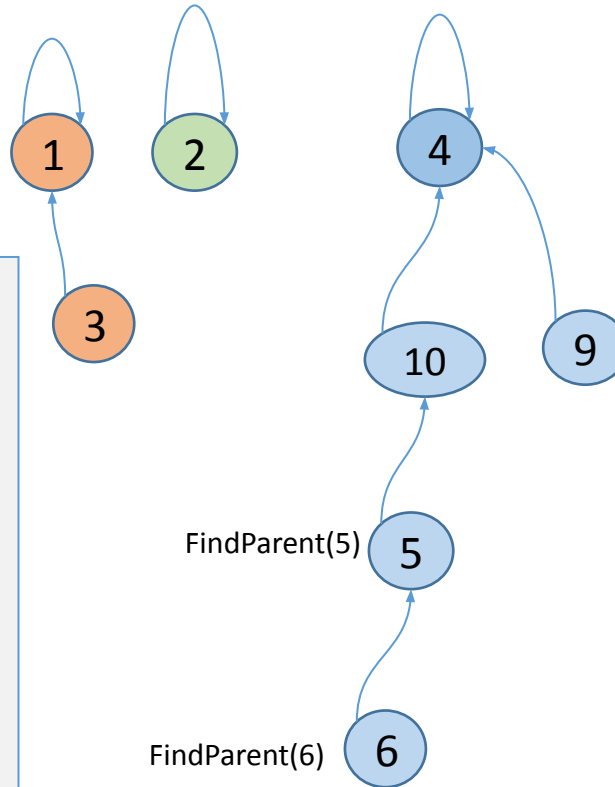
Observe that this is
taking $O(n)$ time
Solution?

```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
parent[a]=x
    return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



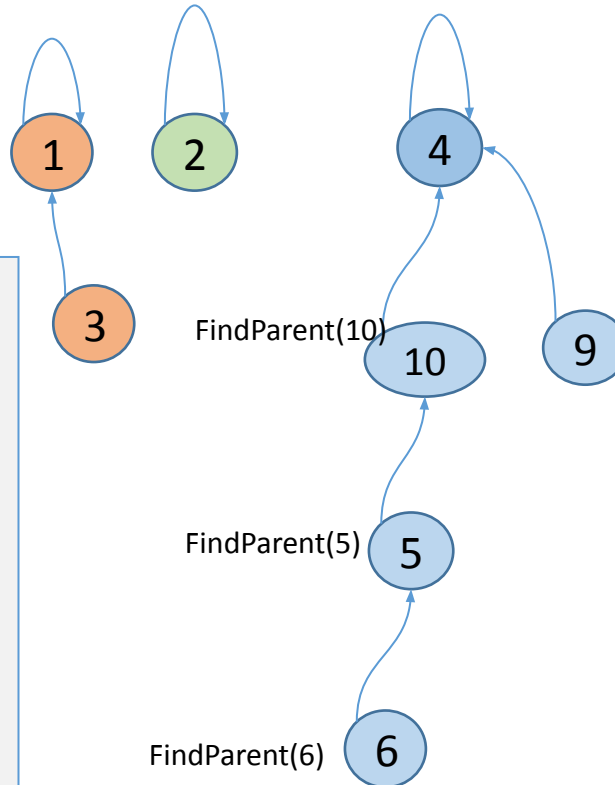
Observe that this is
taking $O(n)$ time
Solution?

```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
parent[a]=x
    return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



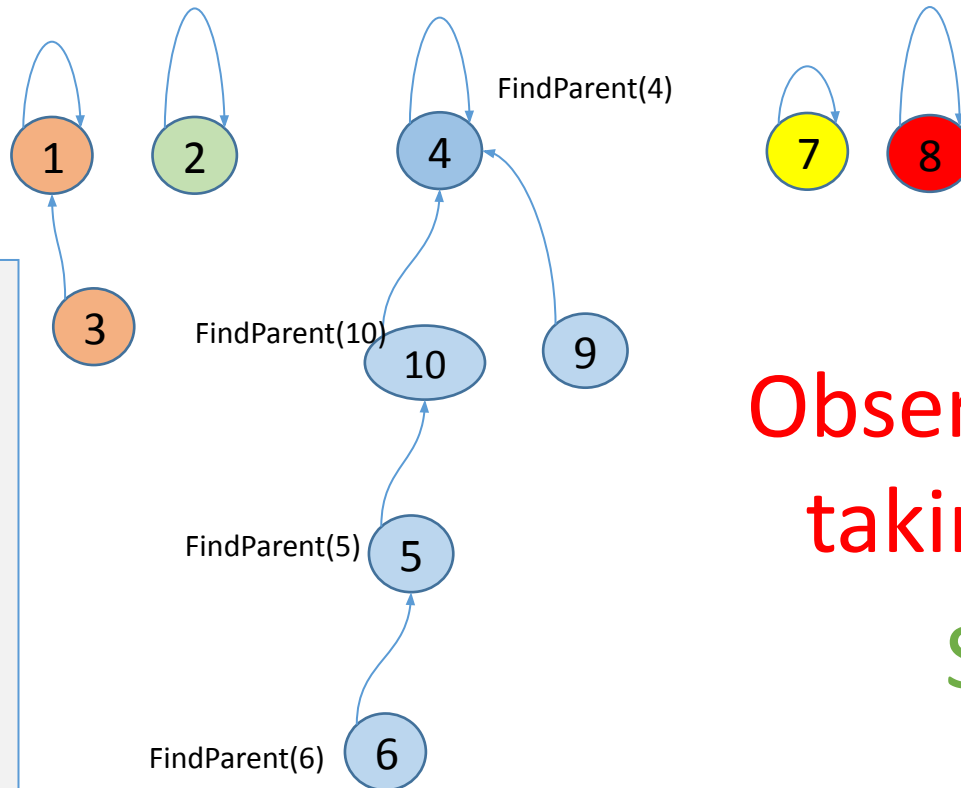
Observe that this is
taking $O(n)$ time
Solution?

```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
parent[a]=x
    return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



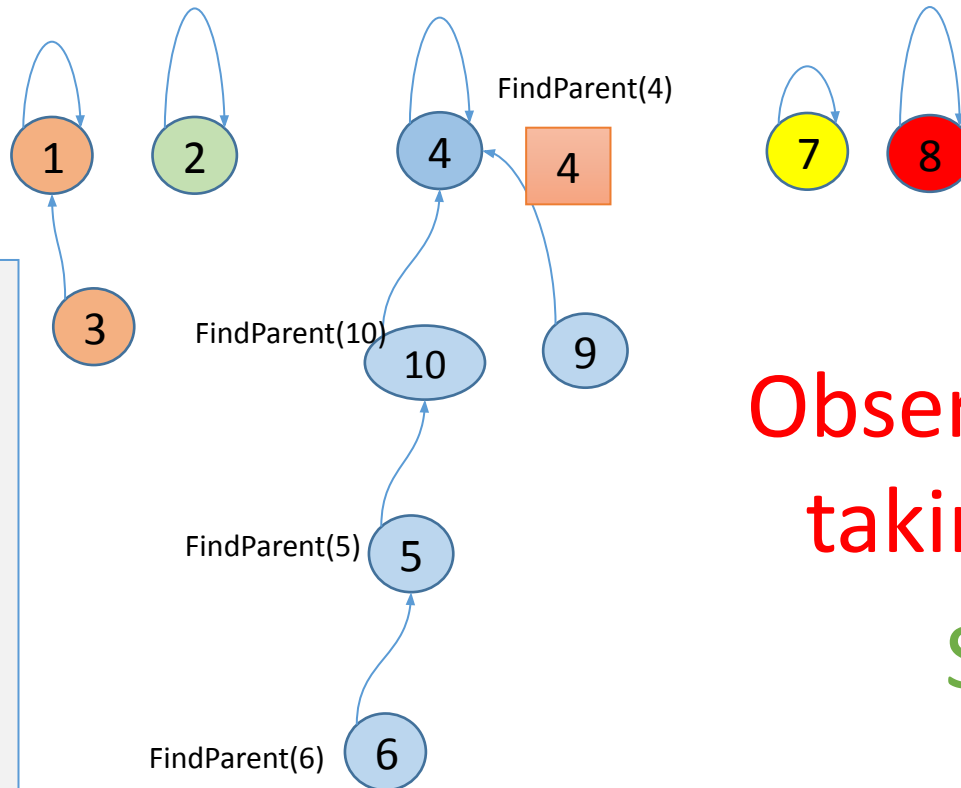
Observe that this is
taking $O(n)$ time
Solution?

```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
parent[a]=x
    return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



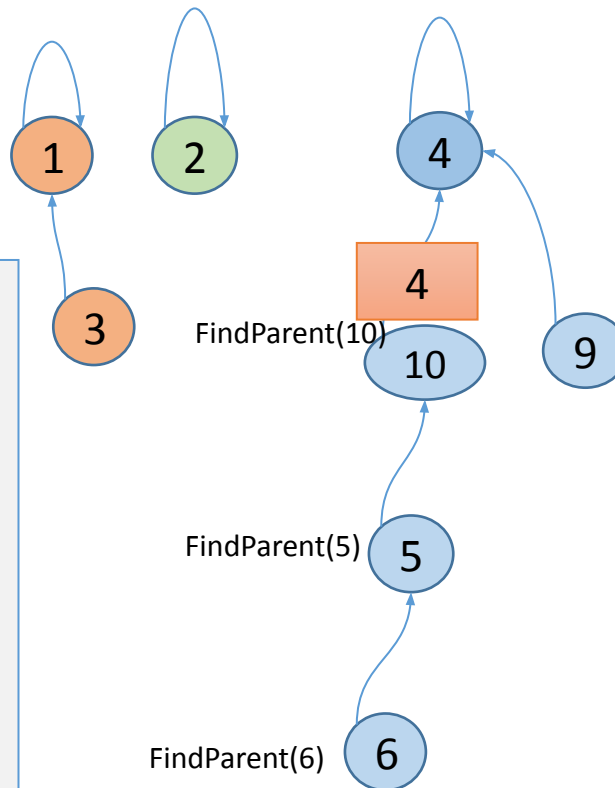
Observe that this is
taking $O(n)$ time
Solution?

```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
parent[a]=x
    return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



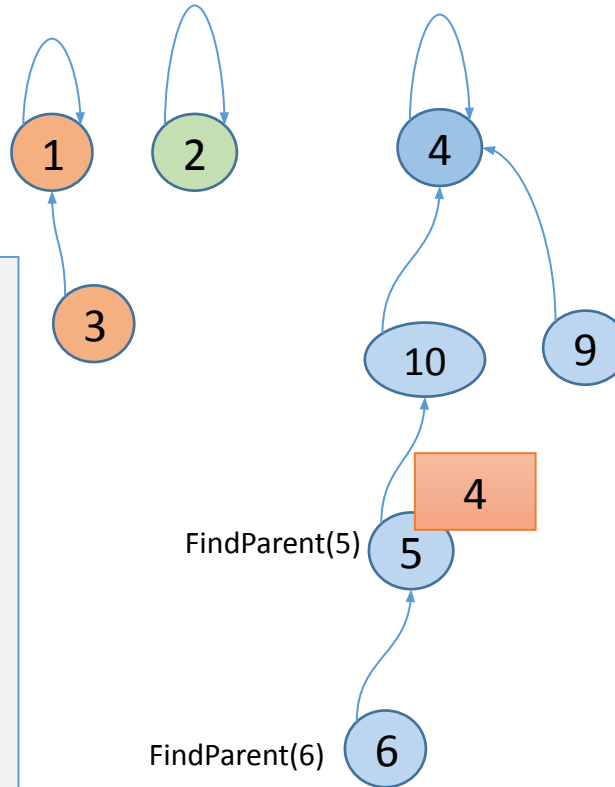
Observe that this is
taking $O(n)$ time
Solution?

```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
parent[a]=x
    return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



Observe that this is
taking $O(n)$ time

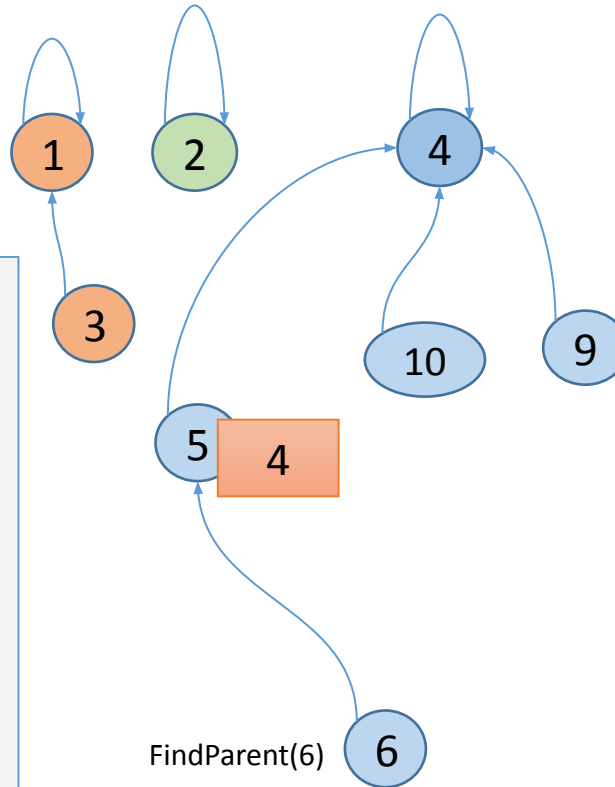
Solution?

```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
parent[a]=x
    return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



Observe that this is
taking $O(n)$ time

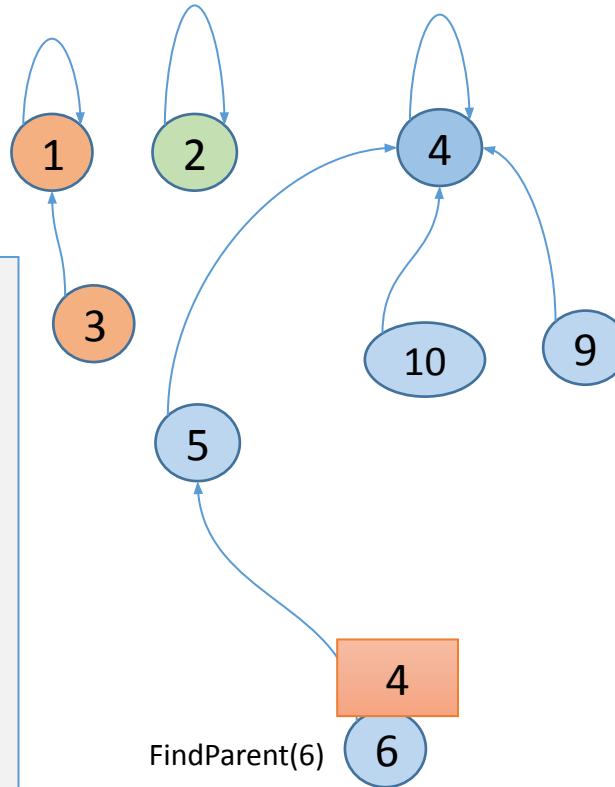
Solution?

```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
parent[a]=x
    return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



Observe that this is
taking $O(n)$ time

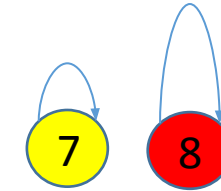
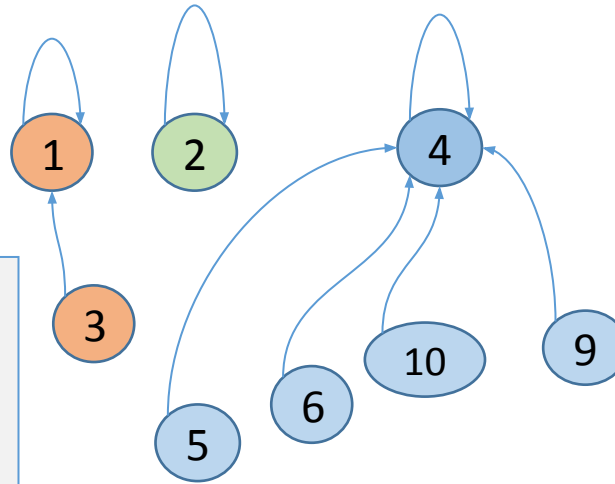
Solution?

```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
parent[a]=x
    return x
```

Set Functions

- FindParent(a)
 - Returns the root.

1	2	3	4	5	6	7	8	9	10
1	2	1	4	10	5	7	8	9	10



```
FindParent (a)
    if parent[a]==a
return a
    else
        x=FindParent (parent[a])
parent[a]=x
    return x
```

Observe that this is
taking $O(n)$ time

Solution?
FOUND!!!

Thank You

